Introduction to Machine Learning

CSA2001-Module-4

What is Machine Learning?

"**Learning** is any process by which a system improves performance from experience."

- Herbert Simon

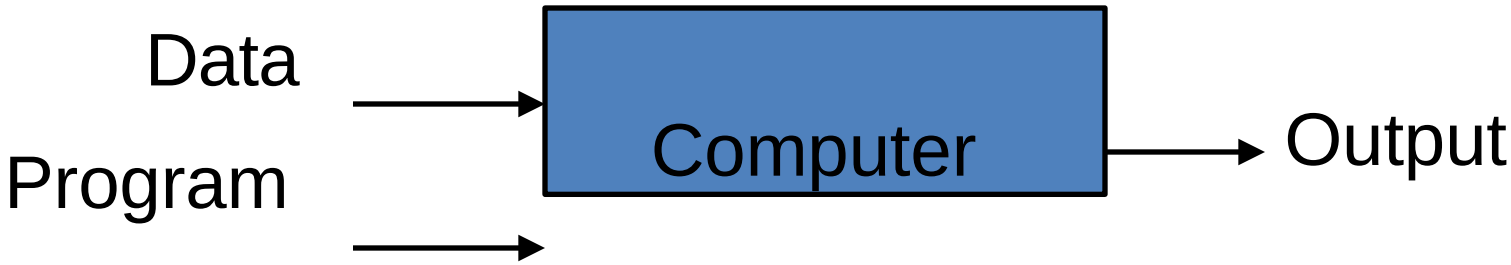
Definition by Tom Mitchell (1998):

Machine Learning is the study of algorithms that

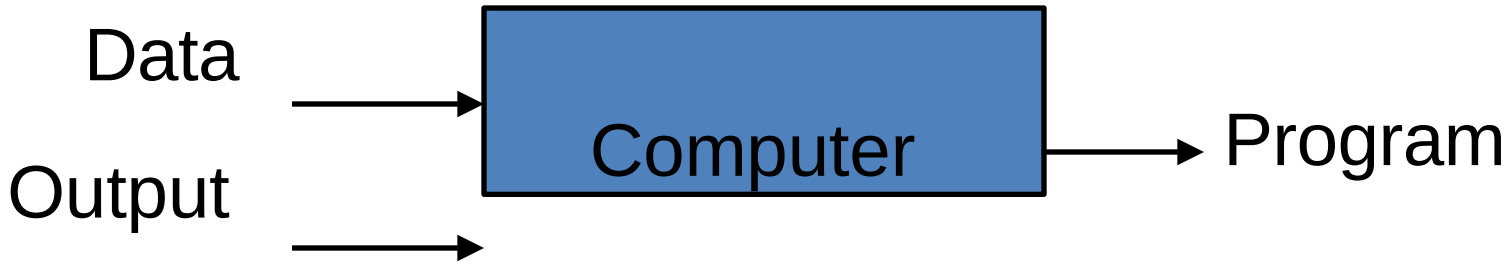
- improve their performance *P*
- at some task *T*
- with experience *E*.

A well-defined learning task is given by <*P*, *T*, *E*>.

Traditional Programming



Machine Learning



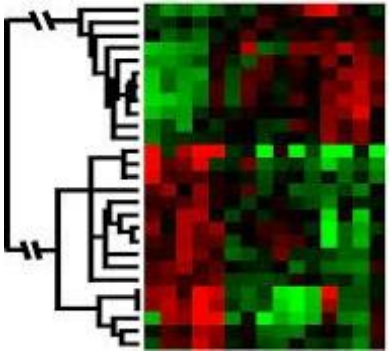
Slide credit: Pedro Domingos

When Do We Use Machine Learning?

ML is used when:

- Human expertise does not exist (navigating on Mars)
- Humans can't explain their expertise (speech recognition)
- Models must be customized (personalized medicine)
- Models are based on huge amounts of data (genomics)



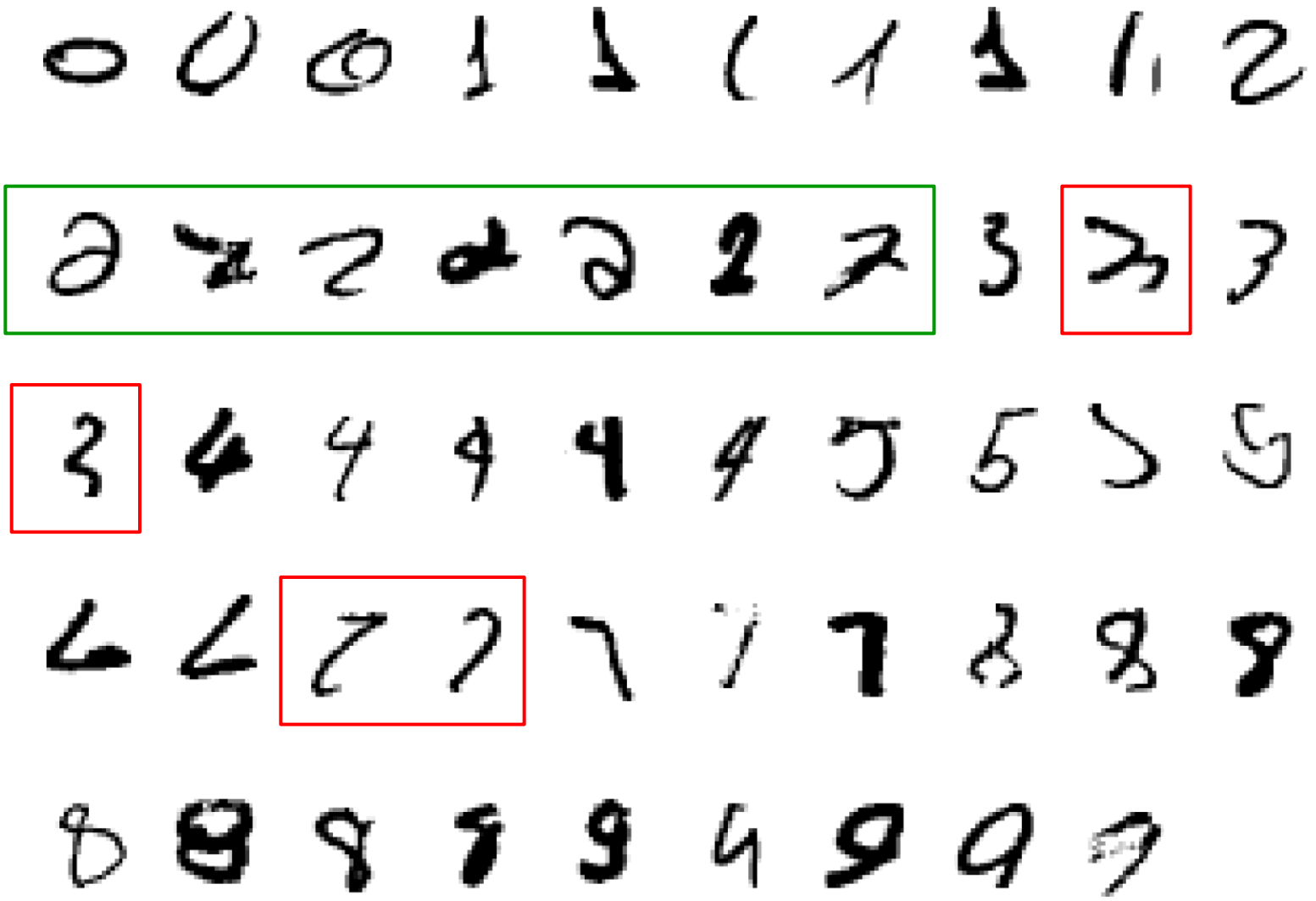



Learning isn't always useful:

- There is no need to "learn" to calculate payroll

Based on slide by E. Alpaydin

A classic example of a task that requires machine learning: It is very hard to say what makes a 2



Slide credit: Geoffrey

Some more examples of tasks that are best solved by using a learning algorithm

- Recognizing patterns:
  - Facial identities or facial expressions
  - Handwritten or spoken words
  - Medical images
- Generating patterns:
  - Generating images or motion sequences
- Recognizing anomalies:
  - Unusual credit card transactions
  - Unusual patterns of sensor readings in a nuclear power plant
- Prediction:
  - Future stock prices or currency exchange rates

Slide credit: Geoffrey

Sample Applications

- Web search
- Computational biology
- Finance
- E-commerce
- Space exploration
- Robotics
- Information extraction
- Social networks
- Debugging software

Slide credit: Pedro

Samuel's Checkers-Player

"Machine Learning: Field of study that gives computers the ability to learn without being explicitly programmed." -Arthur Samuel (1959)



Defining the Learning Task

Improve on task T, with respect to
performance metric P, based on experience E

T: Playing checkers
P: Percentage of games won against an arbitrary opponent
E: Playing practice games against itself

T: Recognizing hand-written words
P: Percentage of words correctly classified
E: Database of human-labeled images of handwritten words

T: Driving on four-lane highways using vision sensors
P: Average distance traveled before a human-judged error
E: A sequence of images and steering commands recorded while observing a human driver.

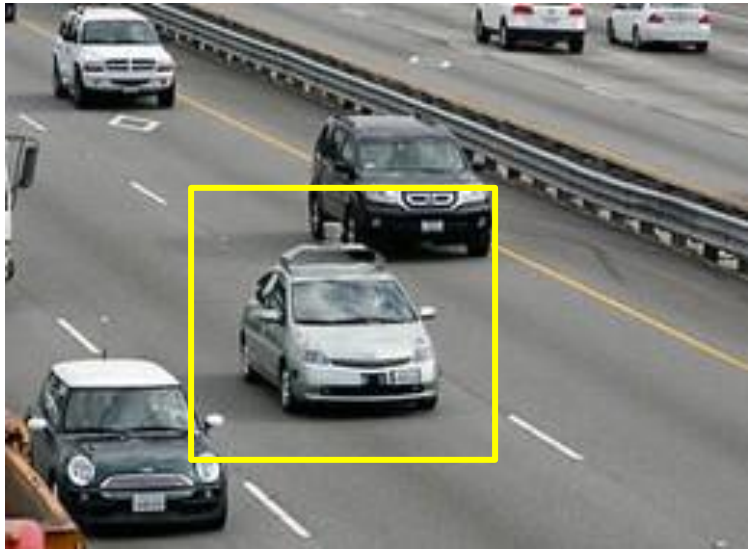
T: Categorize email messages as spam or legitimate.
P: Percentage of email messages correctly classified.
E: Database of emails, some with human-given labels

Slide credit: Ray

State of the Art Applications of Machine Learning

Autonomous C



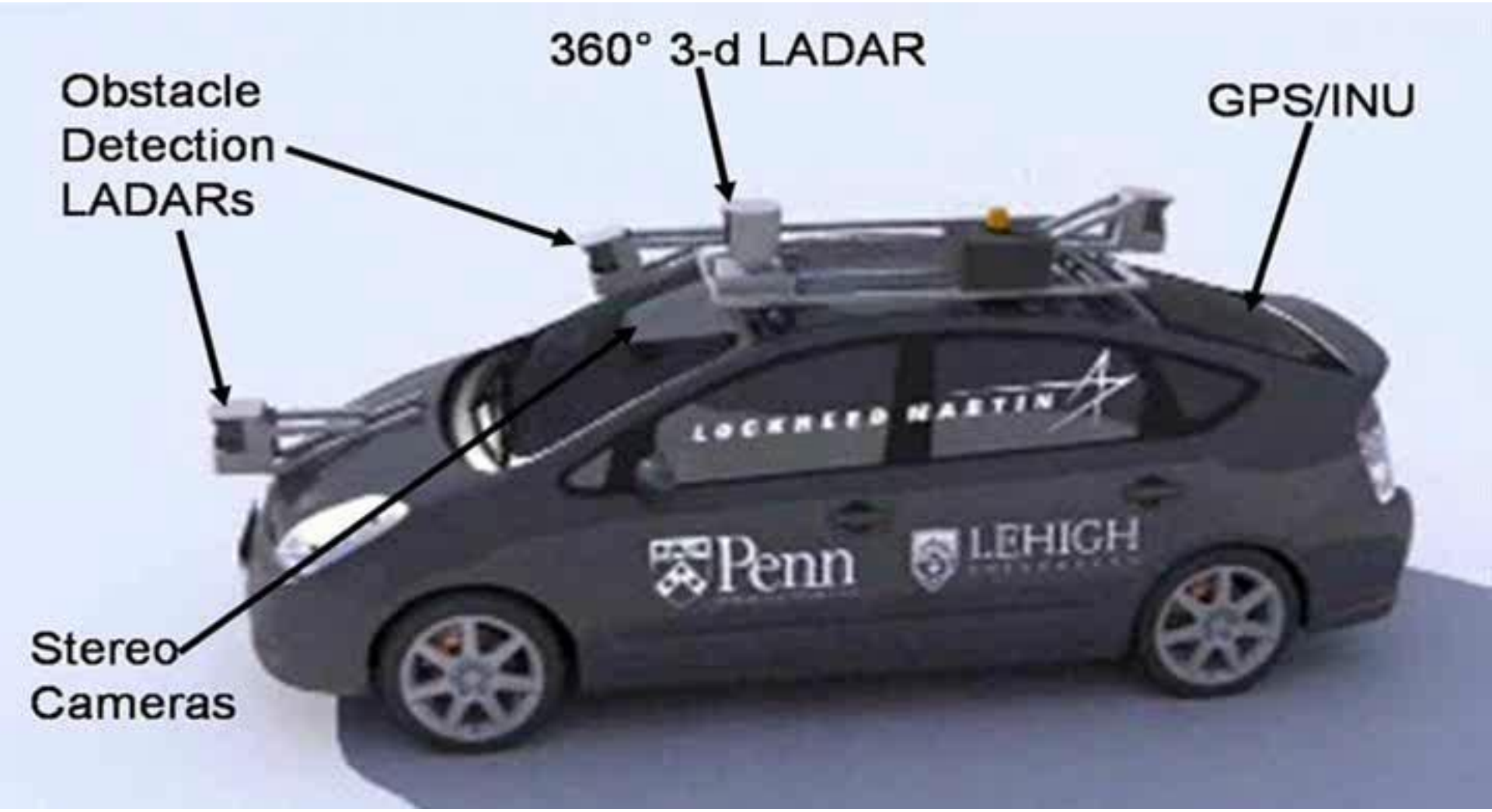


- Nevada made it legal for autonomous cars to drive on roads in June 2011
- As of 2013, four states (Nevada, Florida, California, and Michigan) have legalized autonomous cars

Penn's Autonomous Car →
(Ben Franklin Racing Team)

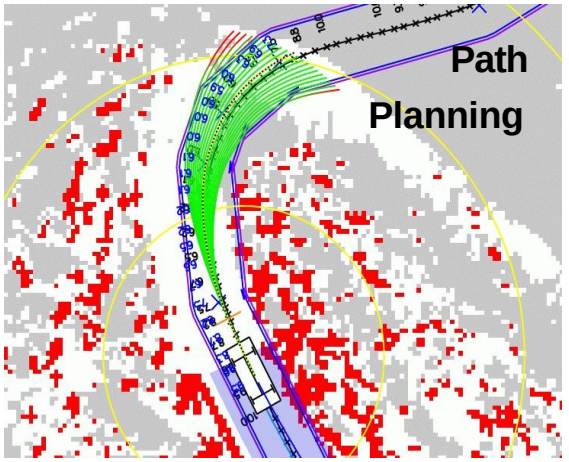


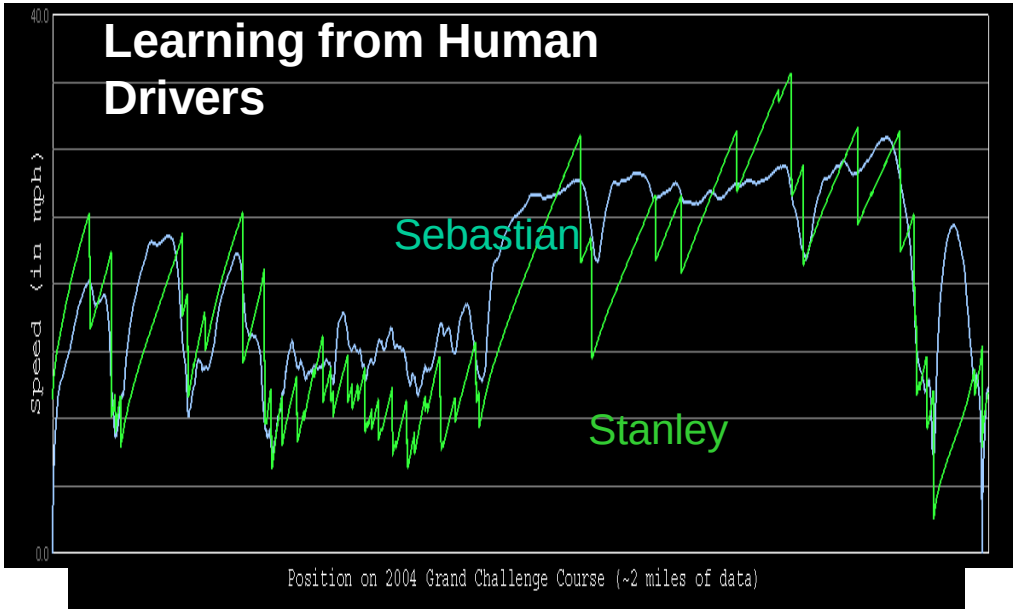
Autonomous Car Sensors



Autonomous Car Technology









Images and movies taken from Sebastian Thrun's multimedia website.

Deep Learning in the Headlines

BUSINESS NEWS

MIT Technology Review

Is Google Cornering the Market on Deep Learning?

A cutting-edge corner of science is being wooed by Silicon Valley, to the dismay of some academics.

By Antonio Regalado on January 29, 2014



How much are a dozen deep-learning researchers worth? Apparently, more than $400 million.

This week, Google reportedly paid that much to acquire DeepMind Technologies, a startup based in



BloombergBusinessweek
Technology

Acquisitions

The Race to Buy the Human Brains Behind Deep Learning Machines

By Ashlee Vance | January 27, 2014

intelligence projects. "DeepMind is bona fide in terms of its research capabilities and depth," says Peter Lee, who heads Microsoft Research.

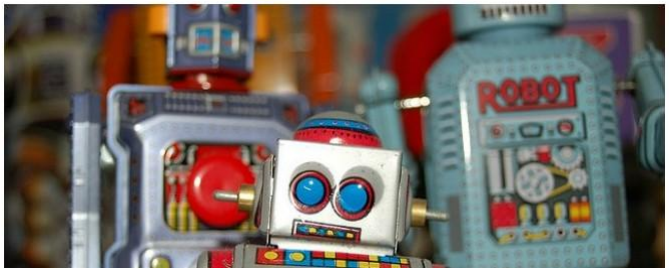
According to Lee, Microsoft, Facebook (FB), and Google find themselves in a battle for deep learning talent. Microsoft has gone from four full-time deep learning experts to 70 in the past three years. "We would have more if the talent was there to

WIRED GEAR SCIENCE ENTERTAINMENT BUSINESS SECURITY DESIG

INNOVATION INSIGHTS | community content | featured

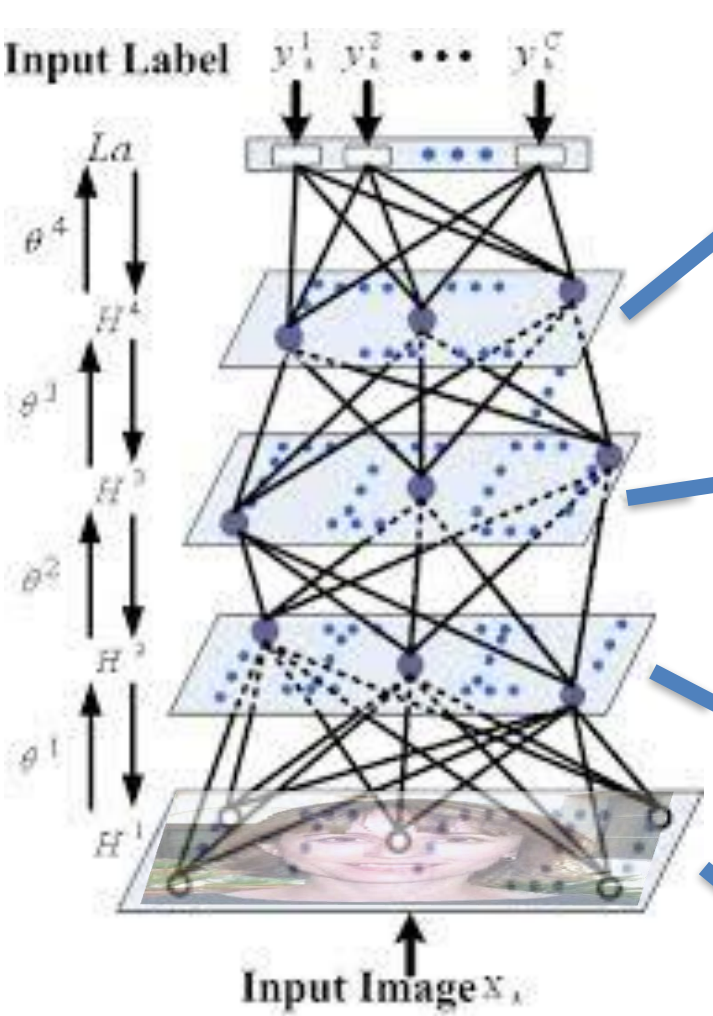
Deep Learning's Role in the Age of Robots

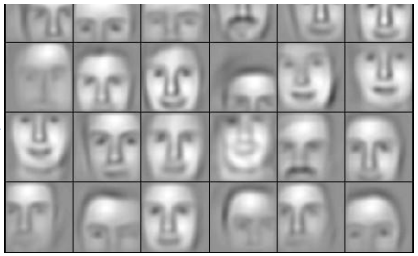
BY JULIAN GREEN, JETPAC 05.02.14 2:56 PM



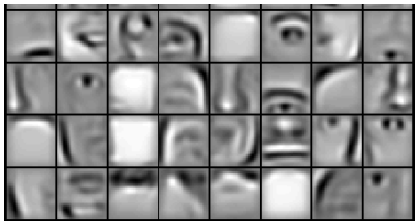


Deep Belief Net on Face Images

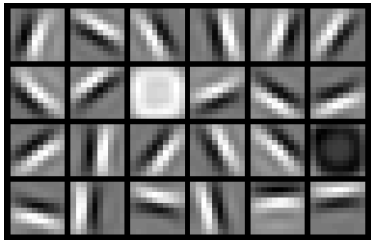




object models



object parts (combination of edges)



edges

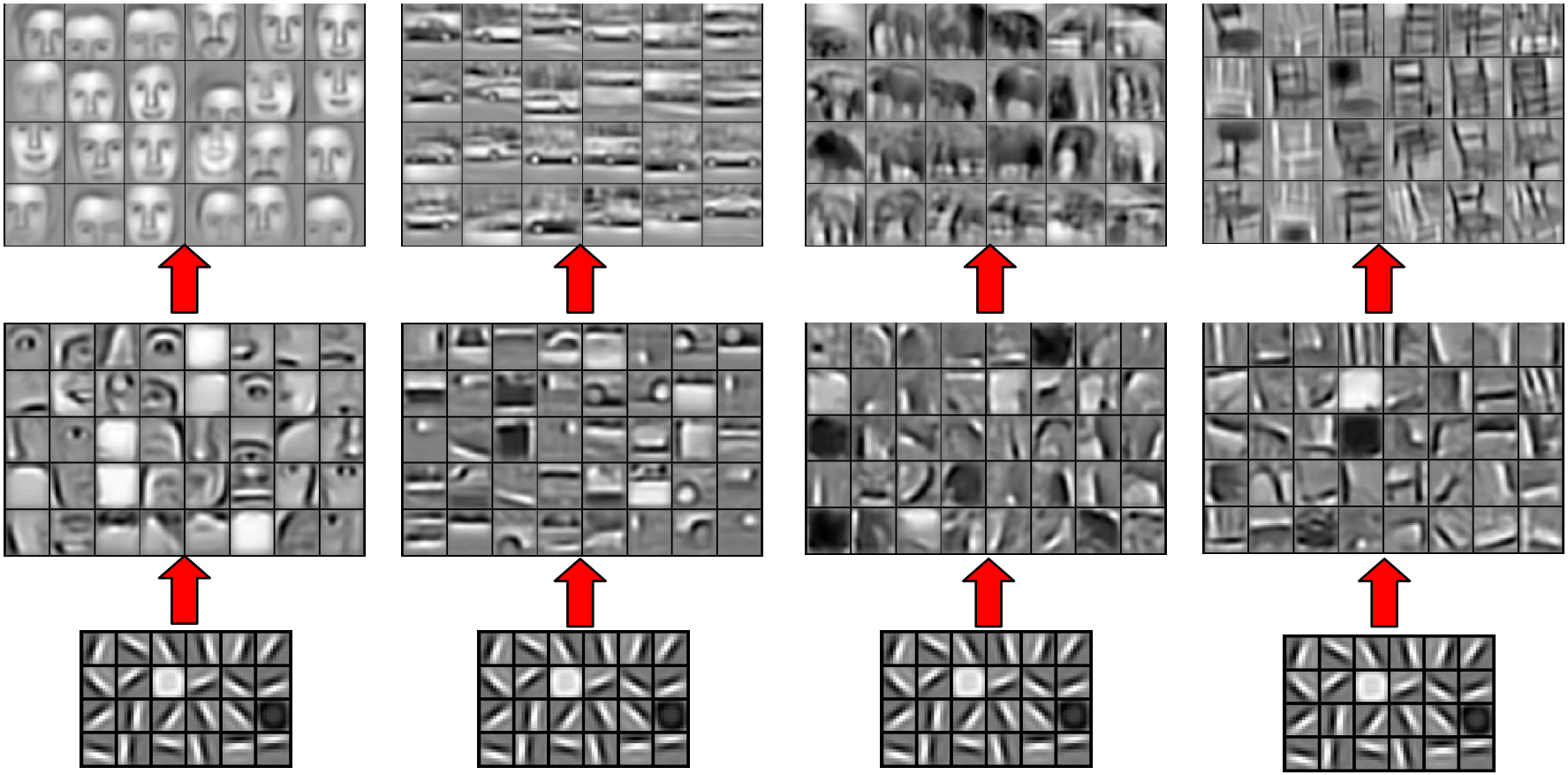


pixels



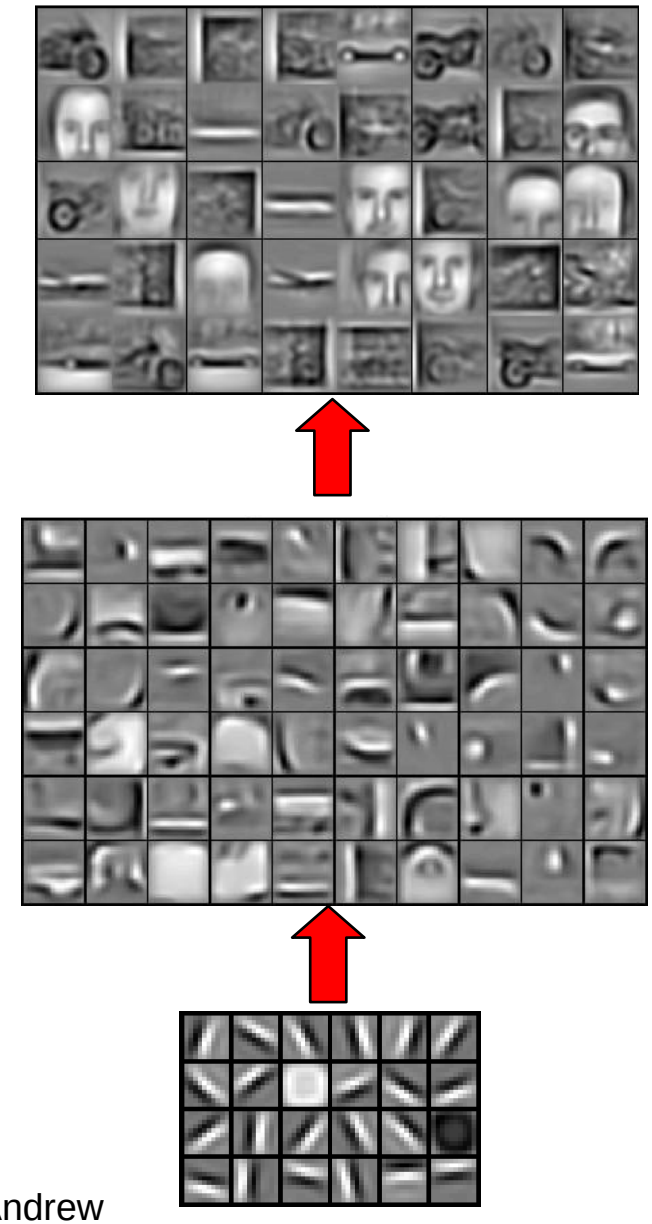
Based on materials by

Learning of Object Parts



Slide credit: Andrew

Training on Multiple Objects



Trained on 4 classes (cars, faces, motorbikes, airplanes).

Second layer: Shared-features and object-specific features.

Third layer: More specific features.

Slide credit: Andrew

Scene Labeling via Deep Learning



[Farabet et al. ICML 2012, PAMI 2013]

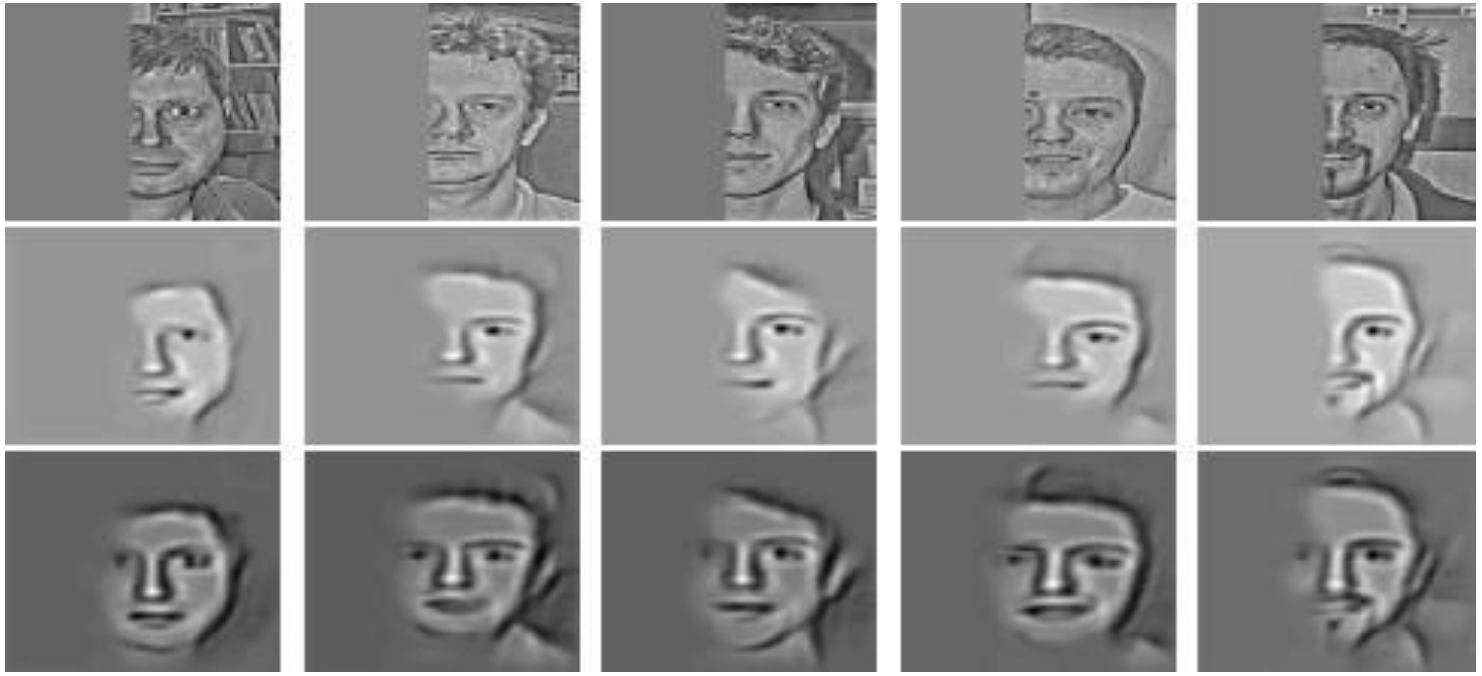
Inference from Deep Learned Models

Generating posterior samples from faces by "filling in" experiments (cf. Lee and Mumford, 2003). Combine bottom-up and top-down inference.

Input images

Samples from feedforward Inference (control)

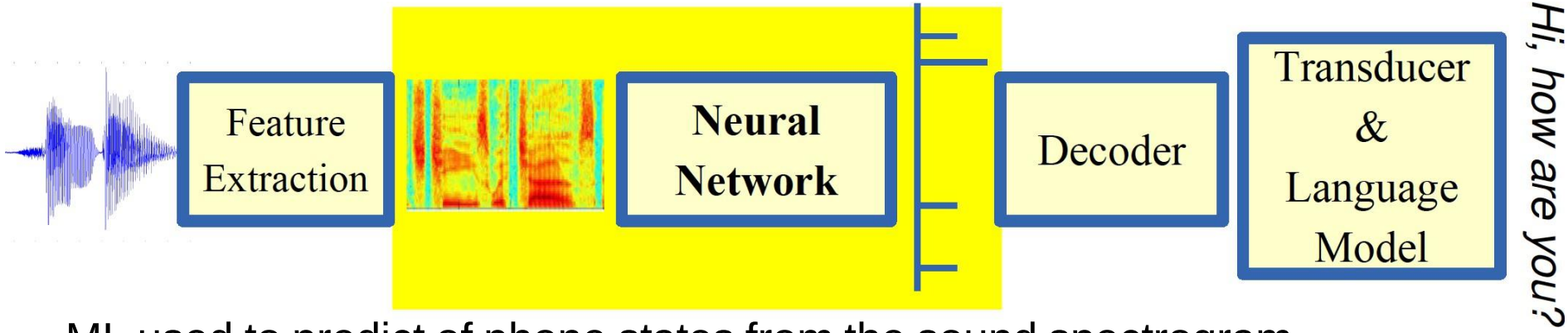
Samples from Full posterior inference



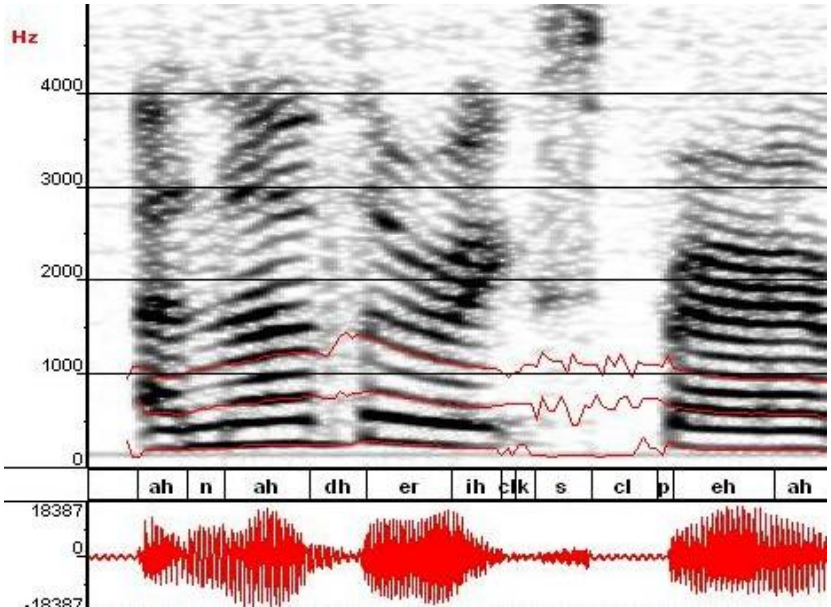
Slide credit: Andrew

Machine Learning in Automatic Speech Recognition

A Typical Speech Recognition System



ML used to predict of phone states from the sound spectrogram



Deep learning has state-of-the-art results

# Hidden Layers	1	2	4	8	10	12
Word Error Rate %	16.0	12.8	11.4	10.9	11.0	11.1

Baseline GMM performance = 15.4%

[Zeiler et al. “On rectified linear units for speech recognition” ICASSP 2013]

Impact of Deep Learning in Speech Technology















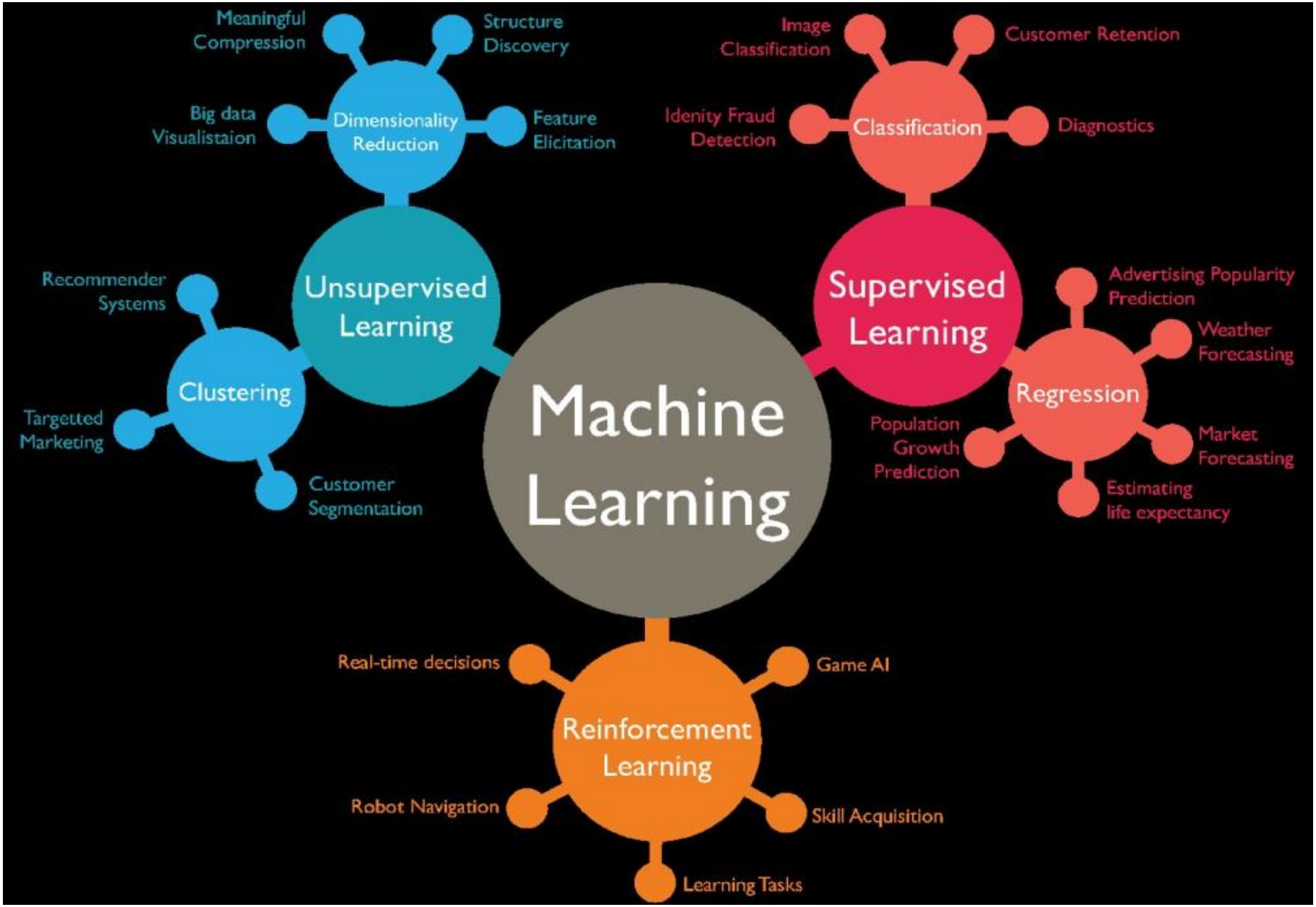
Slide credit: Li Deng, MS

Types of Learning

Types of Learning

- **Supervised (inductive) learning**
  - Given: training data + desired outputs (labels)
- **Unsupervised learning**
  - Given: training data (without desired outputs)
- **Semi-supervised learning**
  - Given: training data + a few desired outputs
- **Reinforcement learning**
  - Rewards from sequence of actions

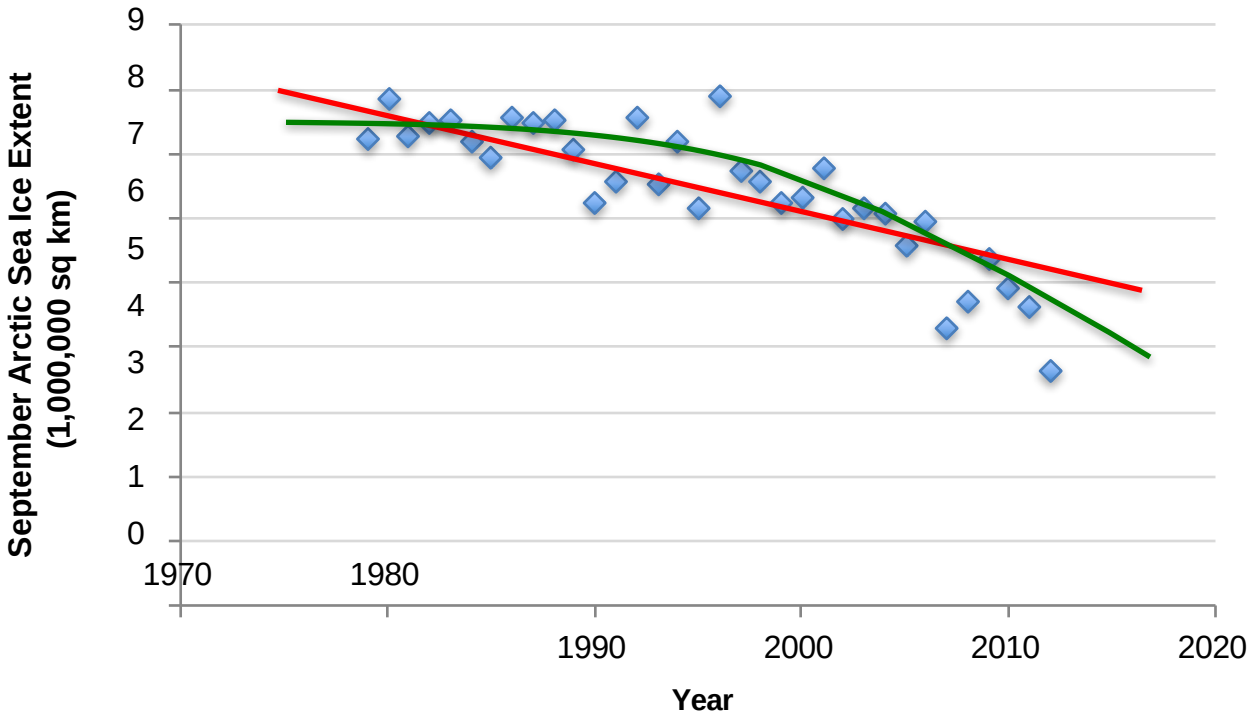
Based on slide by Pedro



Credit:https://towardsdatascience.com/machine-learning-types-2-c1291d4f04b1

Supervised Learning: Regression

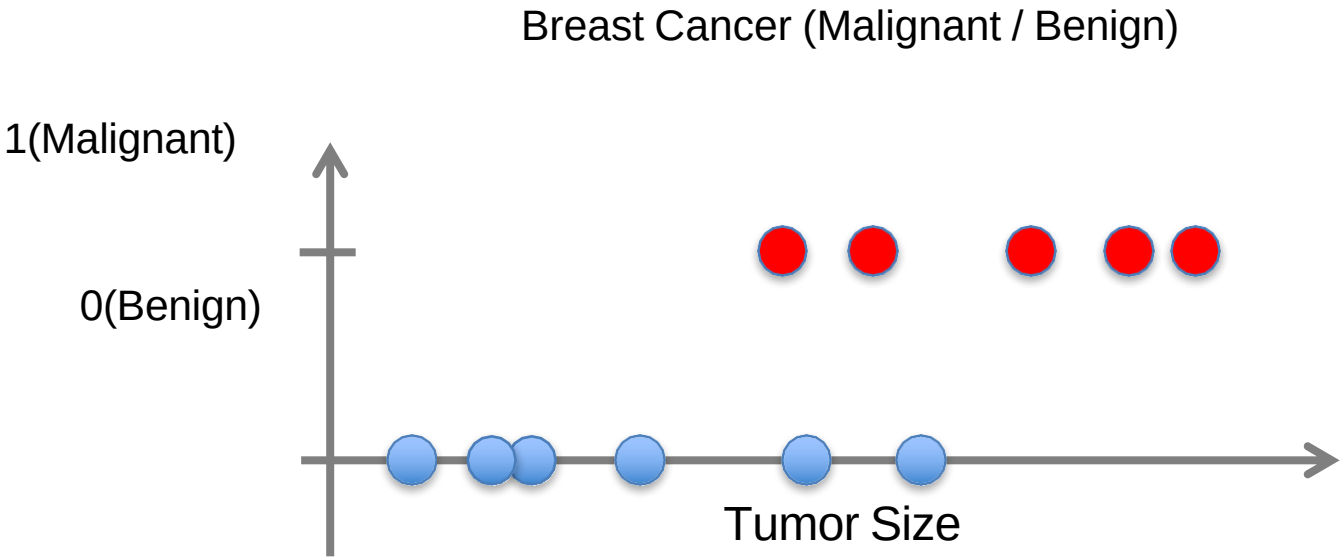
- Given $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$
- Learn a function $f(x)$ to predict y given x
  - y is real-valued == regression



Data from G. Witt. Journal of Statistics Education, Volume 21, Number 1 (2013)

Supervised Learning: Classification

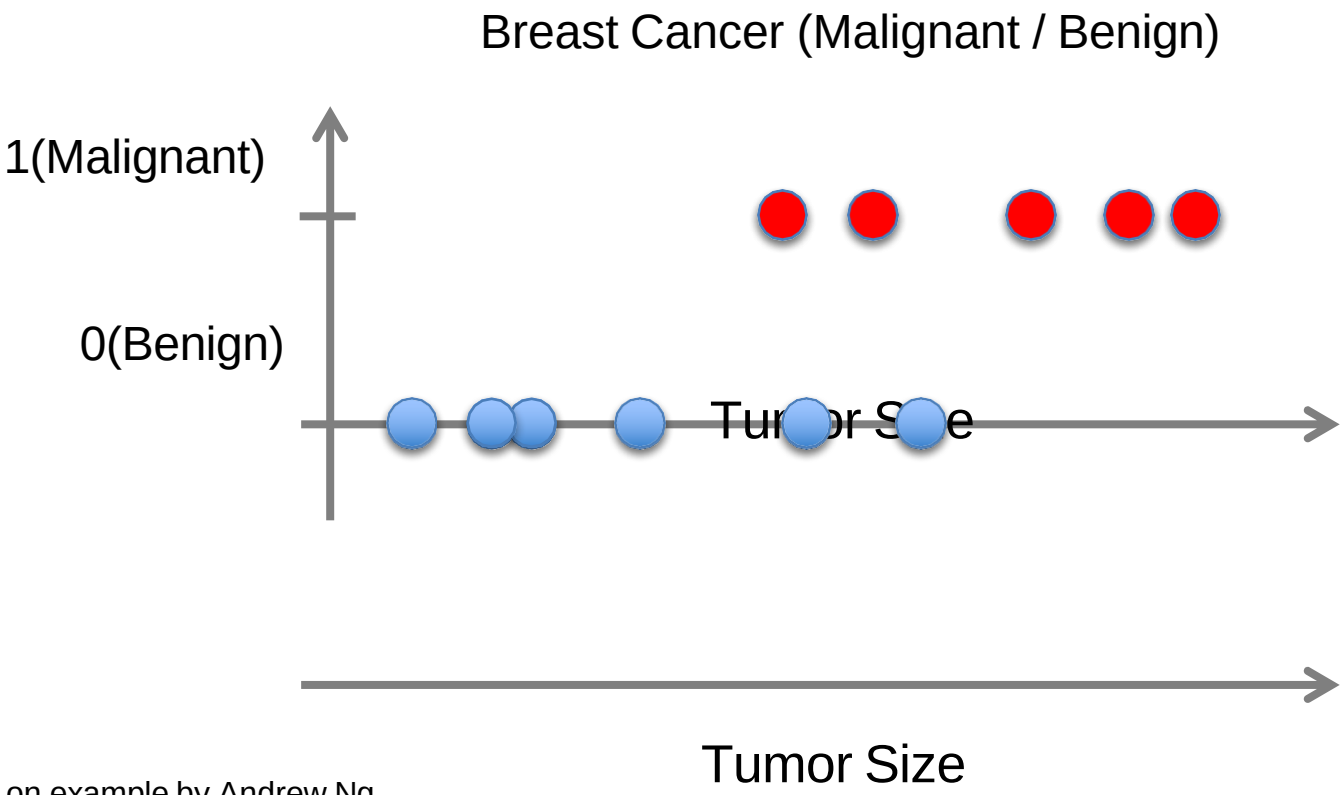
- Given (x_1, y_1), (x_2, y_2), ..., (x_n, y_n)
- Learn a function $f(x)$ to predict y given x
  - y is categorical == classification



Based on example by Andrew Ng

Supervised Learning: Classification

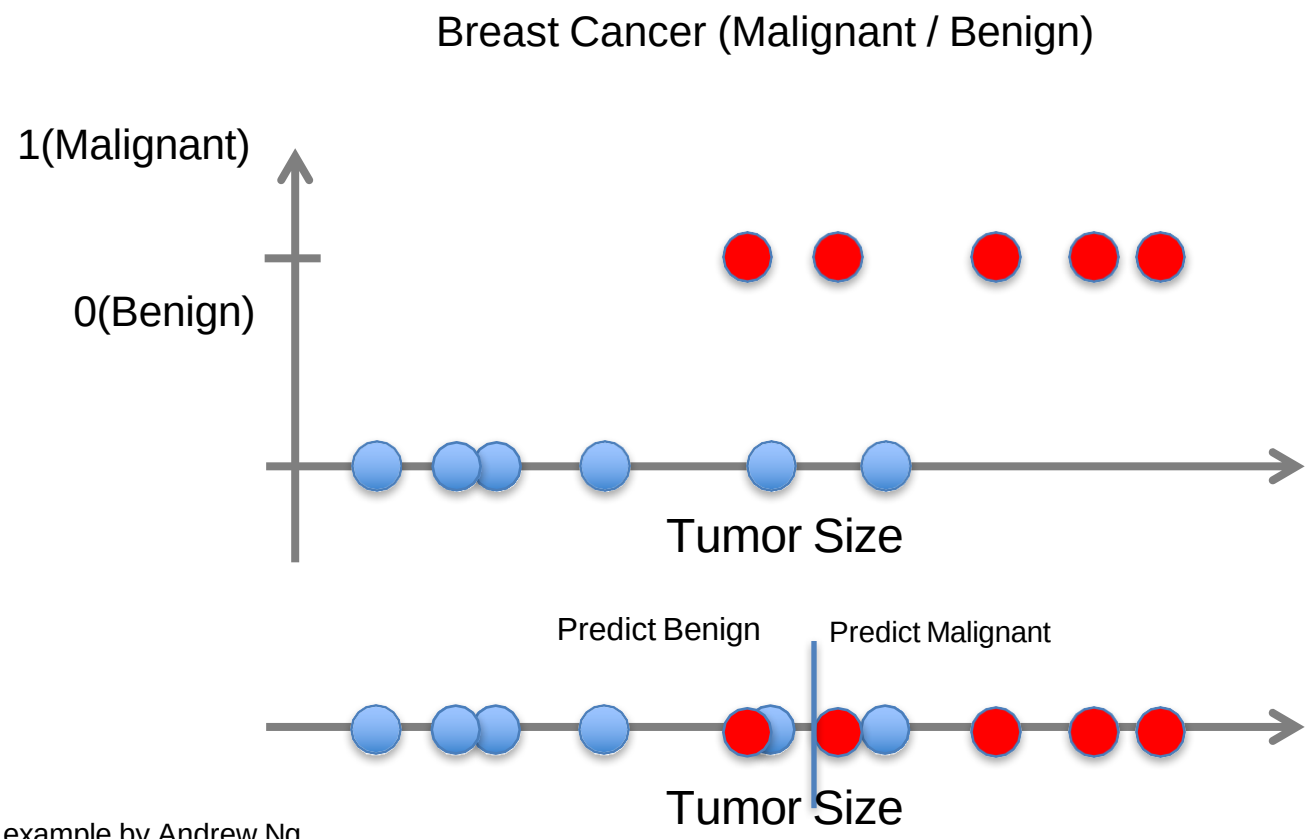
- Given (x_1, y_1), (x_2, y_2), ..., (x_n, y_n)
- Learn a function $f(x)$ to predict y given x
  - y is categorical == classification



Based on example by Andrew Ng

Supervised Learning: Classification

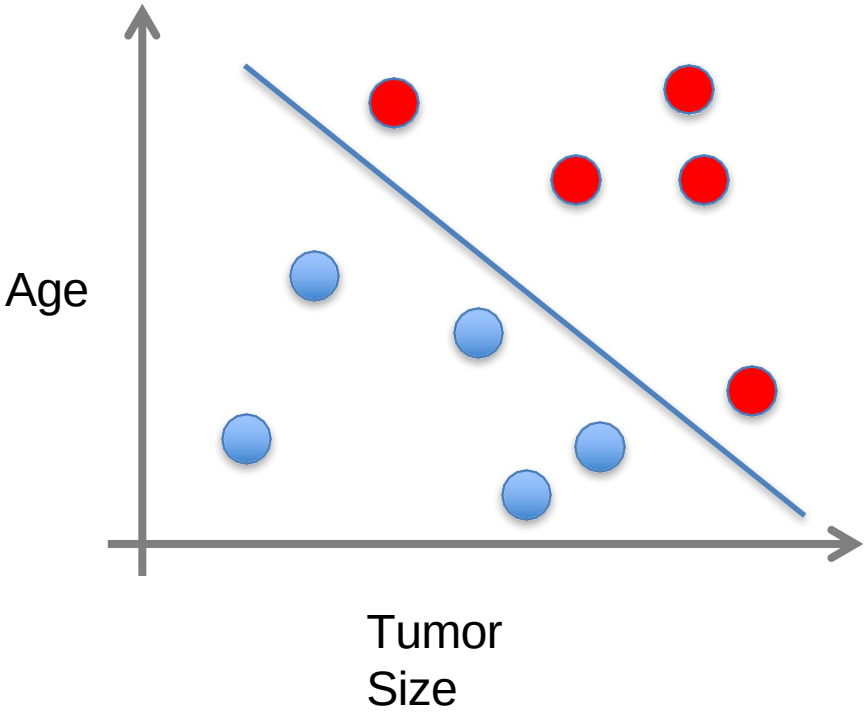
- Given (x_1, y_1), (x_2, y_2), ..., (x_n, y_n)
- Learn a function $f(x)$ to predict y given x
  - y is categorical == classification



Based on example by Andrew Ng

Supervised Learning

- *x* can be multi-dimensional
  - Each dimension corresponds to an attribute



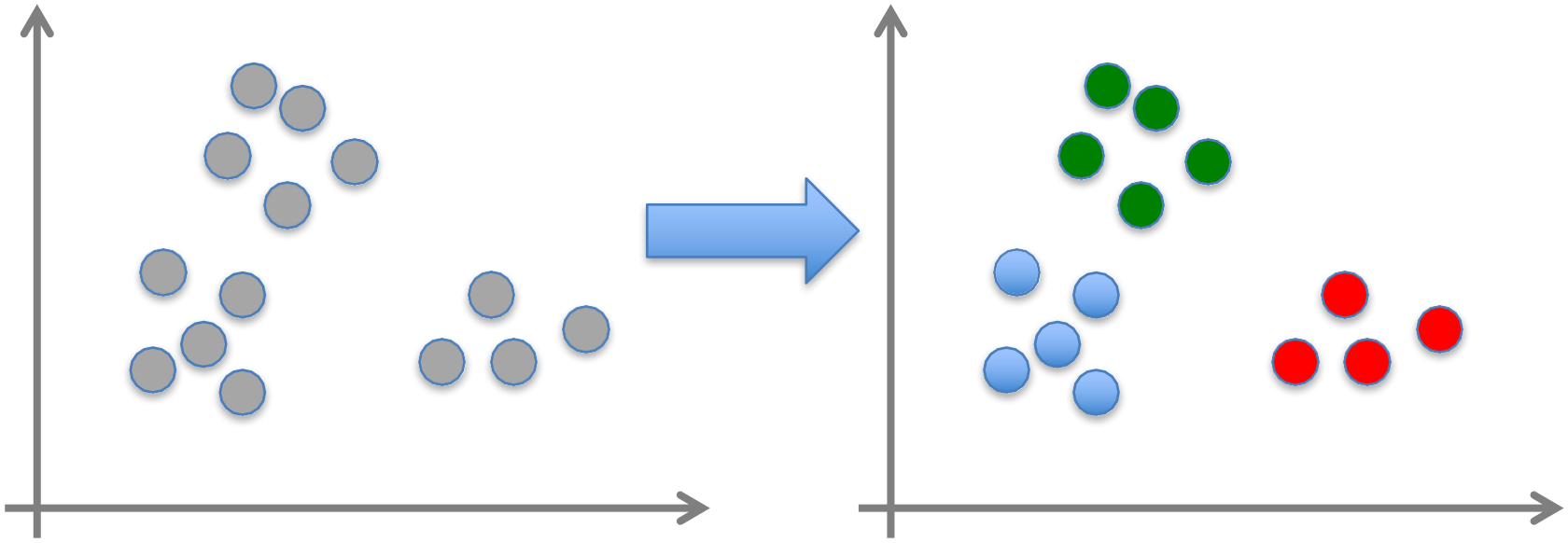
- Clump Thickness
- Uniformity of Cell Size
- Uniformity of Cell Shape

…

Based on example by Andrew Ng

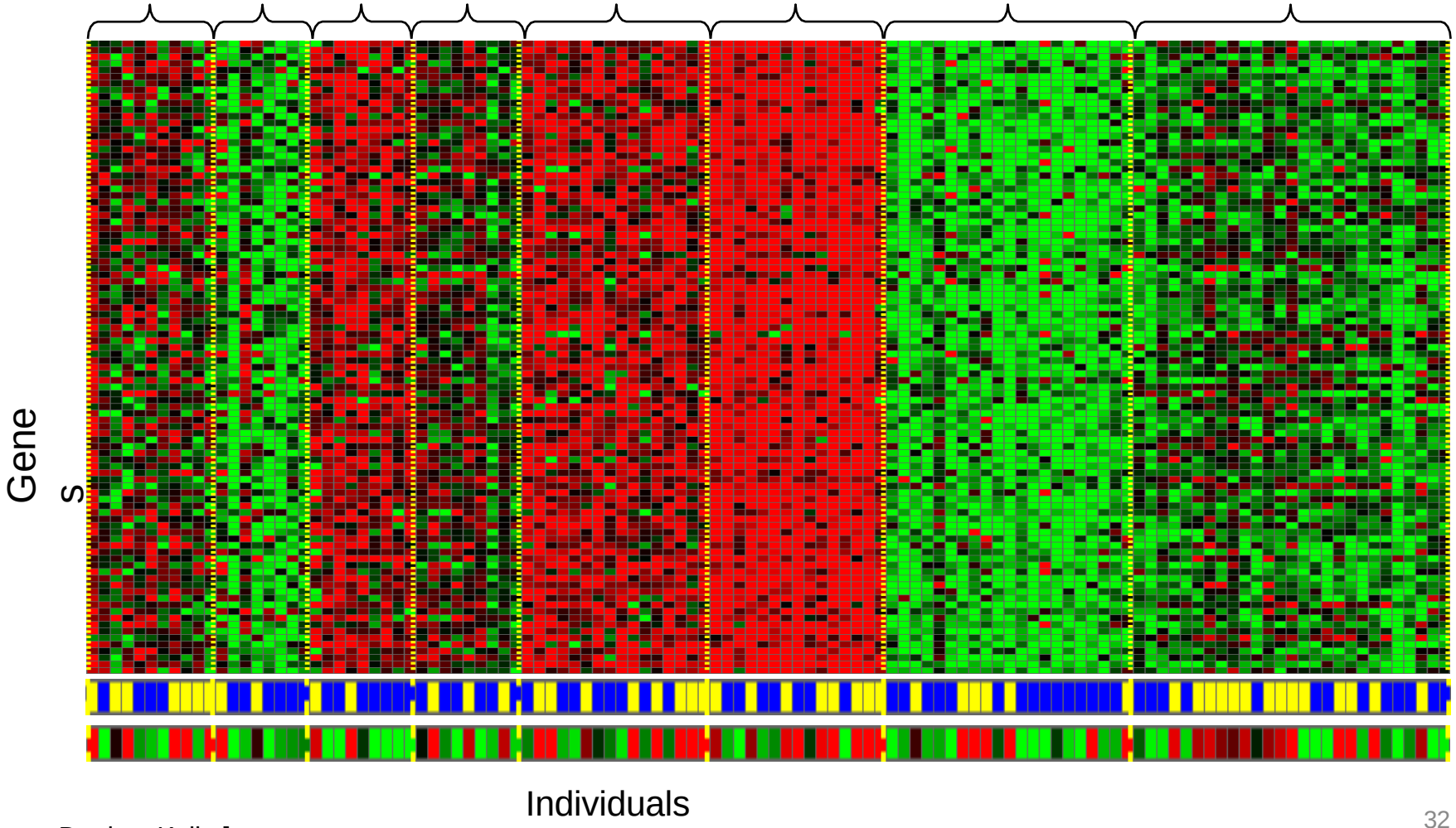
Unsupervised Learning

- Given $x_1, x_2, \ldots, x_n$ (without labels)
- Output hidden structure behind the x's
  - E.g., clustering



Unsupervised Learning

Genomics application: group individuals by genetic similarity

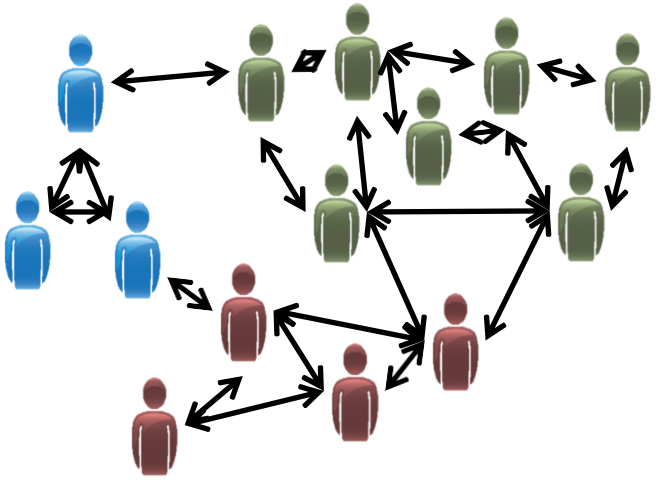


[Source: Daphne Koller]

Unsupervised Learning



Organize computing clusters



Social network analysis



Market segmentation



Astronomical data analysis

Slide credit: Andrew Ng

Unsupervised Learning

- Independent component analysis – separate a combined signal into its original sources

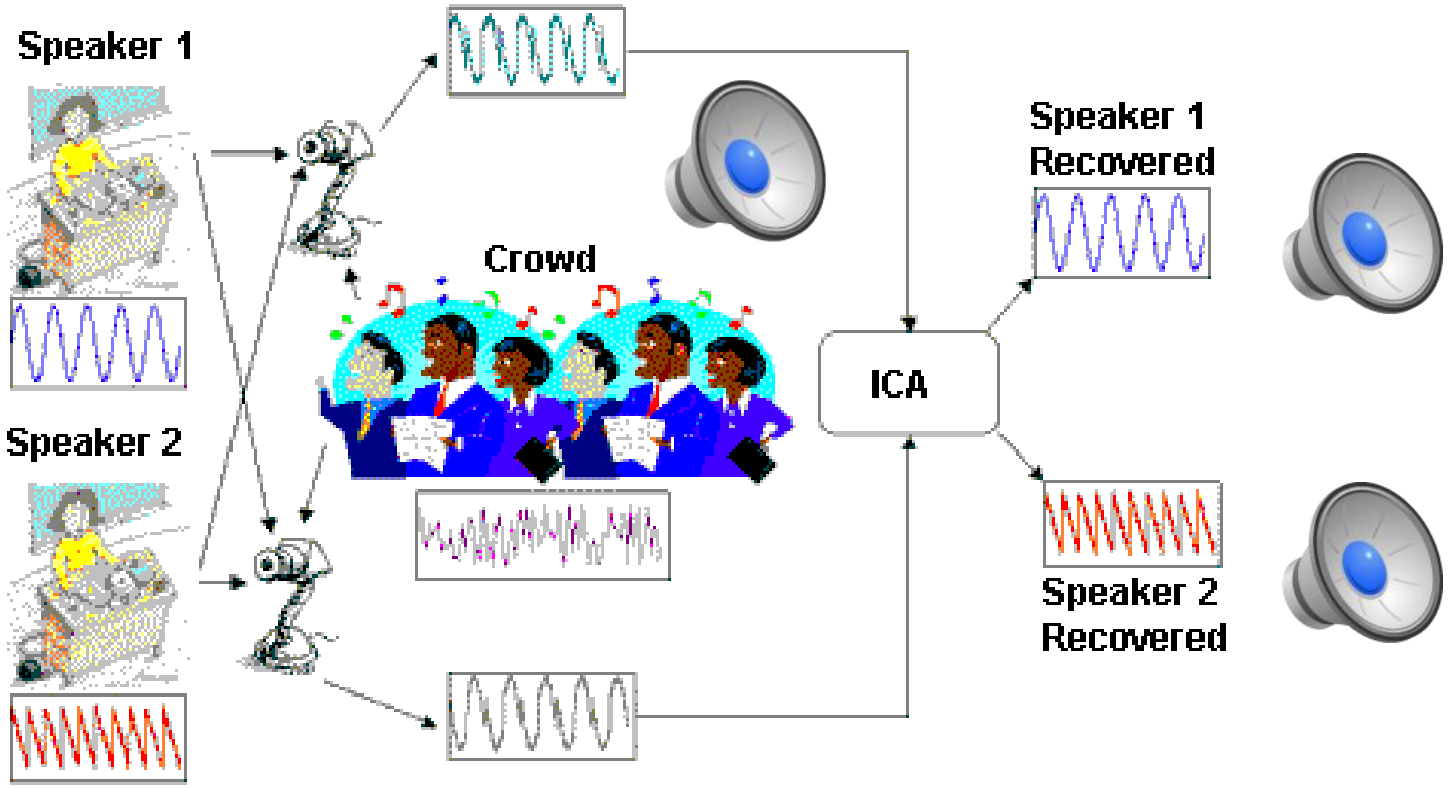


Image credit: statsoft.com Audio from http://www.ism.ac.jp/~shiro/research/blindsep.html

Unsupervised Learning

- Independent component analysis – separate a combined signal into its original sources

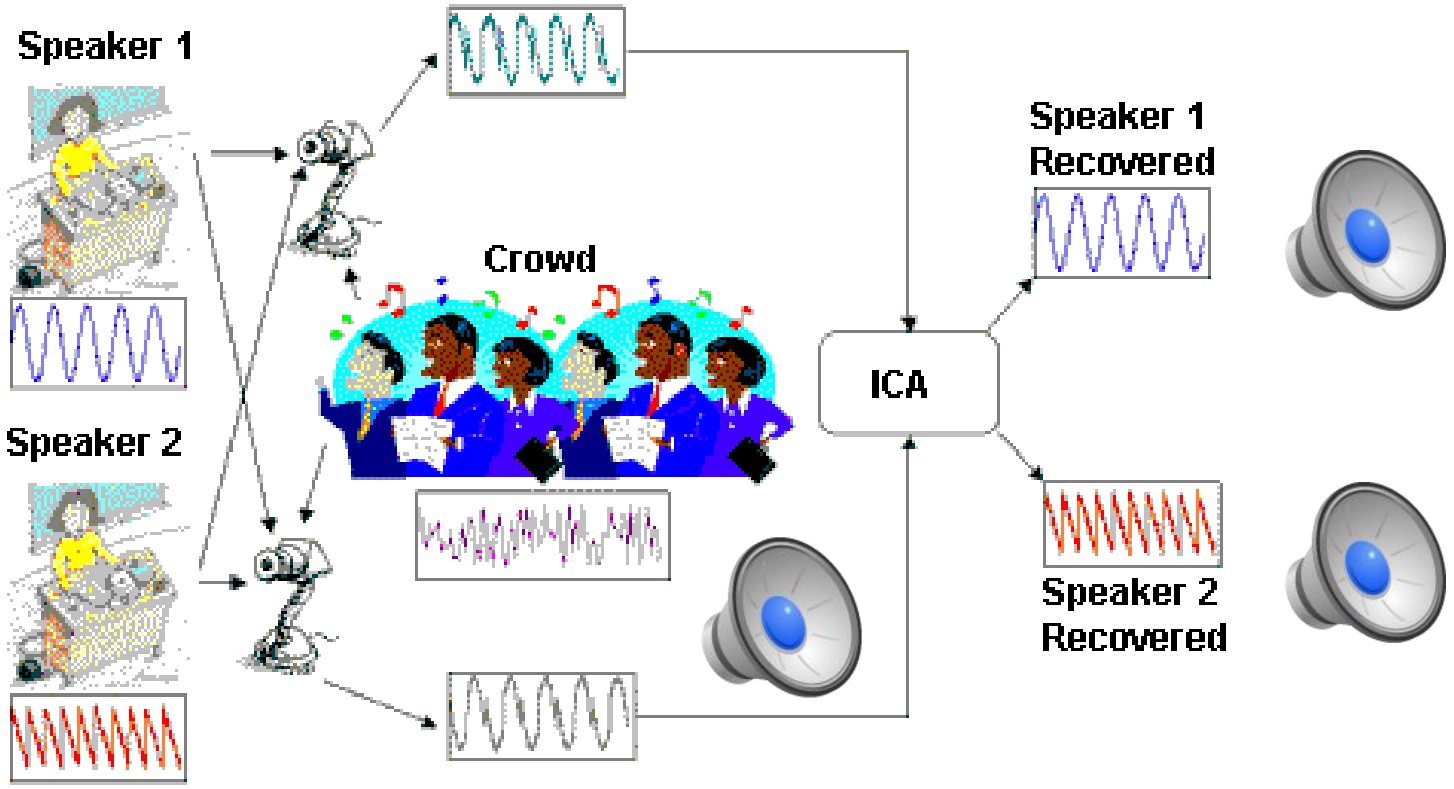


Image credit: statsoft.com Audio from http://www.ism.ac.jp/~shiro/research/blindsep.html

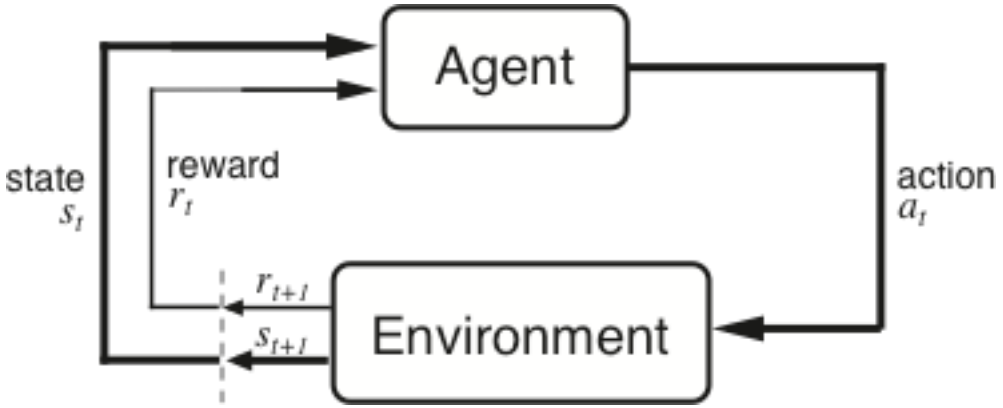
Semi-supervised learning

Semi-supervised learning is a learning problem that involves a small number of labelled examples and a large number of unlabeled examples.
Learning problems of this type are challenging as neither supervised nor unsupervised learning algorithms are able to make effective use of the mixtures of labelled and untellable data.

Reinforcement Learning

- Given a sequence of states and actions with (delayed) rewards, output a policy
  - Policy is a mapping from states → actions that tells you what to do in a given state
- Examples:
  - Credit assignment problem
  - Game playing
  - Robot in a maze
  - Balance a pole on your hand

The Agent-Environment Interface



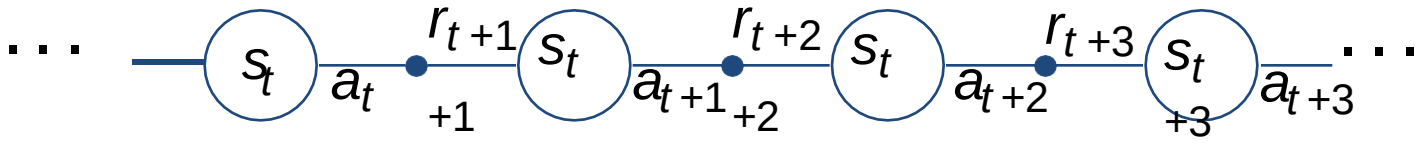
Agent and environment interact at discrete time steps : $t = 0, 1, 2, \mathrm{K}$

Agent observes state at step t: $s_t \in S$

produces action at step t: $a_t \in A(s_t)$

gets resulting reward : $r_{t+1} \in \Re$

and resulting next state : s_{t+1}



Slide credit: Sutton &

Reinforcement Learning



https://www.youtube.com/watch?v=4cgWya-wjgY

Inverse Reinforcement Learning

- Learn policy from user demonstrations

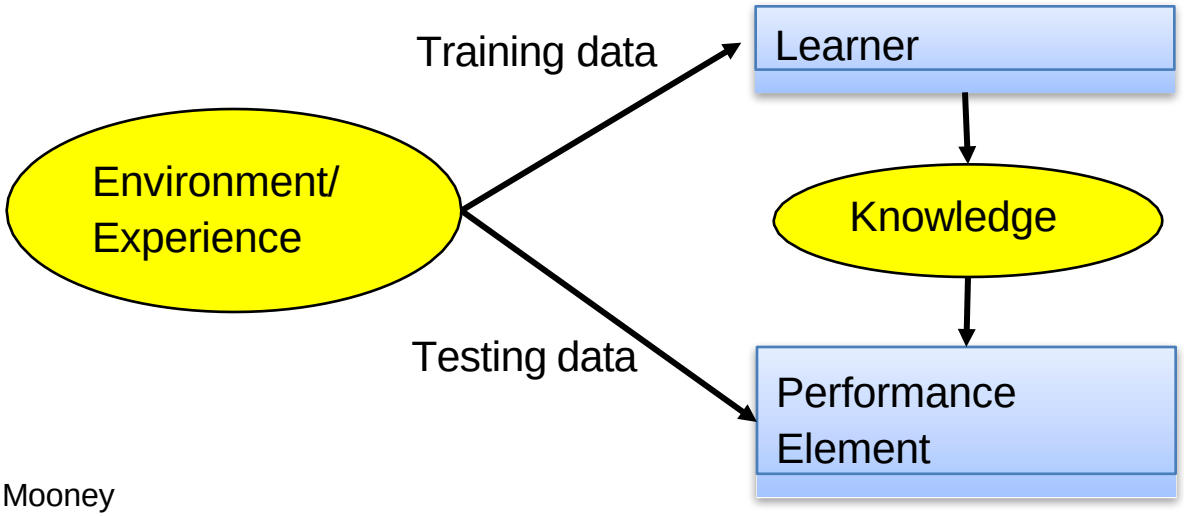


Stanford Autonomous Helicopter http://heli.stanford.edu/
https://www.youtube.com/watch?v=VCdxqn0fcnE

Framing a Learning Problem

Designing a Learning System

- Choose the training experience
- Choose exactly what is to be learned
  - i.e. the ***target function***
- Choose how to represent the target function
- Choose a learning algorithm to infer the target function from the experience



Based on slide by Ray Mooney

ML in a Nutshell

- Tens of thousands of machine learning algorithms
  - Hundreds new every year

- Every ML algorithm has three components:
  - **Representation**
  - **Optimization**
  - **Evaluation**

Slide credit: Pedro Domingos

Various Function Representations

- Numerical functions
  - Linear regression
  - Neural networks
  - Support vector machines
- Symbolic functions
  - Decision trees
  - Rules in propositional logic
  - Rules in first-order predicate logic
- Instance-based functions
  - Nearest-neighbor
  - Case-based
- Probabilistic Graphical Models
  - Naïve Bayes
  - Bayesian networks
  - Hidden-Markov Models (HMMs)
  - Probabilistic Context Free Grammars (PCFGs)
  - Markov networks

Slide credit: Ray Mooney

Various Search/Optimization Algorithms

- Gradient descent
  - Perceptron
  - Backpropagation
- Dynamic Programming
  - HMM Learning
  - PCFG Learning
- Divide and Conquer
  - Decision tree induction
  - Rule learning
- Evolutionary Computation
  - Genetic Algorithms (GAs)
  - Genetic Programming (GP)
  - Neuro-evolution

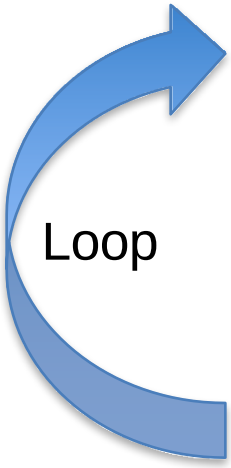
Slide credit: Ray Mooney

Evaluation

- Accuracy
- Precision and recall
- Squared error
- Likelihood
- Posterior probability
- Cost / Utility
- Margin
- Entropy
- K-L divergence
- etc.

Slide credit: Pedro Domingos

ML in Practice



- Understand domain, prior knowledge, and goals
- Data integration, selection, cleaning, pre-processing, etc.
- Learn models
- Interpret results
- Consolidate and deploy discovered knowledge

Based on a slide by Pedro Domingos

A Brief History of Machine Learning

History of Machine Learning

- 1950s
  - Samuel's checker player
  - Selfridge's Pandemonium
- 1960s:
  - Neural networks: Perceptron
  - Pattern recognition
  - Learning in the limit theory
  - Minsky and Papert prove limitations of Perceptron
- 1970s:
  - Symbolic concept induction
  - Winston's arch learner
  - Expert systems and the knowledge acquisition bottleneck
  - Quinlan's ID3
  - Michalski's AQ and soybean diagnosis
  - Scientific discovery with BACON
  - Mathematical discovery with AM

Slide credit: Ray

History of Machine Learning (cont.)

- 1980s:
  - Advanced decision tree and rule learning
  - Explanation-based Learning (EBL)
  - Learning and planning and problem solving
  - Utility problem
  - Analogy
  - Cognitive architectures
  - Resurgence of neural networks (connectionism, backpropagation)
  - Valiant's PAC Learning Theory
  - Focus on experimental methodology
- 1990s
  - Data mining
  - Adaptive software agents and web applications
  - Text learning
  - Reinforcement learning (RL)
  - Inductive Logic Programming (ILP)
  - Ensembles: Bagging, Boosting, and Stacking
  - Bayes Net learning

Slide credit: Ray

History of Machine Learning (cont.)

- 2000s
  - Support vector machines & kernel methods
  - Graphical models
  - Statistical relational learning
  - Transfer learning
  - Sequence labeling
  - Collective classification and structured outputs
  - Computer Systems Applications (Compilers, Debugging, Graphics, Security)
  - E-mail management
  - Personalized assistants that learn
  - Learning in robotics and vision
- 2010s
  - Deep learning systems
  - Learning for big data
  - Bayesian methods
  - Multi-task & lifelong learning
  - Applications to vision, speech, social networks, learning to read, etc.

Based on slide by Ray

Perspectives And Issues In Machine Learning

Which algorithms perform best for which types of problems and representations?

How much training data is sufficient?

What algorithms exist for learning general target functions from specific training examples ?

How does the number of training examples influence accuracy ?

When and how can prior knowledge held by the learner guide the process of generalizing from examples ?

What is the best strategy for choosing a useful next training experience, and how does the choice of this strategy alter the complexity of the learning problem ?

What is the best way to reduce the learning task to one or more function approximation problems ?

How can the learner automatically alter its representation to improve its ability to represent and learn the target function ?

K-Means Clustering

(Help: https://www.javatpoint.com/k-means-clustering-algorithm-in-machine-learning)

K-Means Clustering Statement

K-means tries to partition x data points into the set of k clusters where each data point is assigned to its closest cluster. This method is defined by the objective function which tries to minimize the sum of all squared distances within a cluster, for all clusters.

The objective function is defined as:

$$\arg\min_{\mathbf{S}} \sum_{i=1}^{k} \left(\sum_{\mathbf{x}_j \in S_i} \|\mathbf{x}_j - \boldsymbol{\mu}_i\|^2 \right)$$

Where xj is a data point in the data set, Si is a cluster (set of data points and ui is the cluster mean(the center of cluster of Si)

K-Means Clustering Algorithm:

1. Choose a value of k, number of clusters to be formed.

2. Randomly select k data points from the data set as the intital cluster centeroids/centers

3. For each datapoint:

a. Compute the distance between the datapoint and the cluster centroid

b. Assign the datapoint to the closest centroid

4. For each cluster calculate the new mean based on the datapoints in the cluster.

5. Repeat 3 & 4 steps until mean of the clusters stops changing or maximum number of iterations reached.

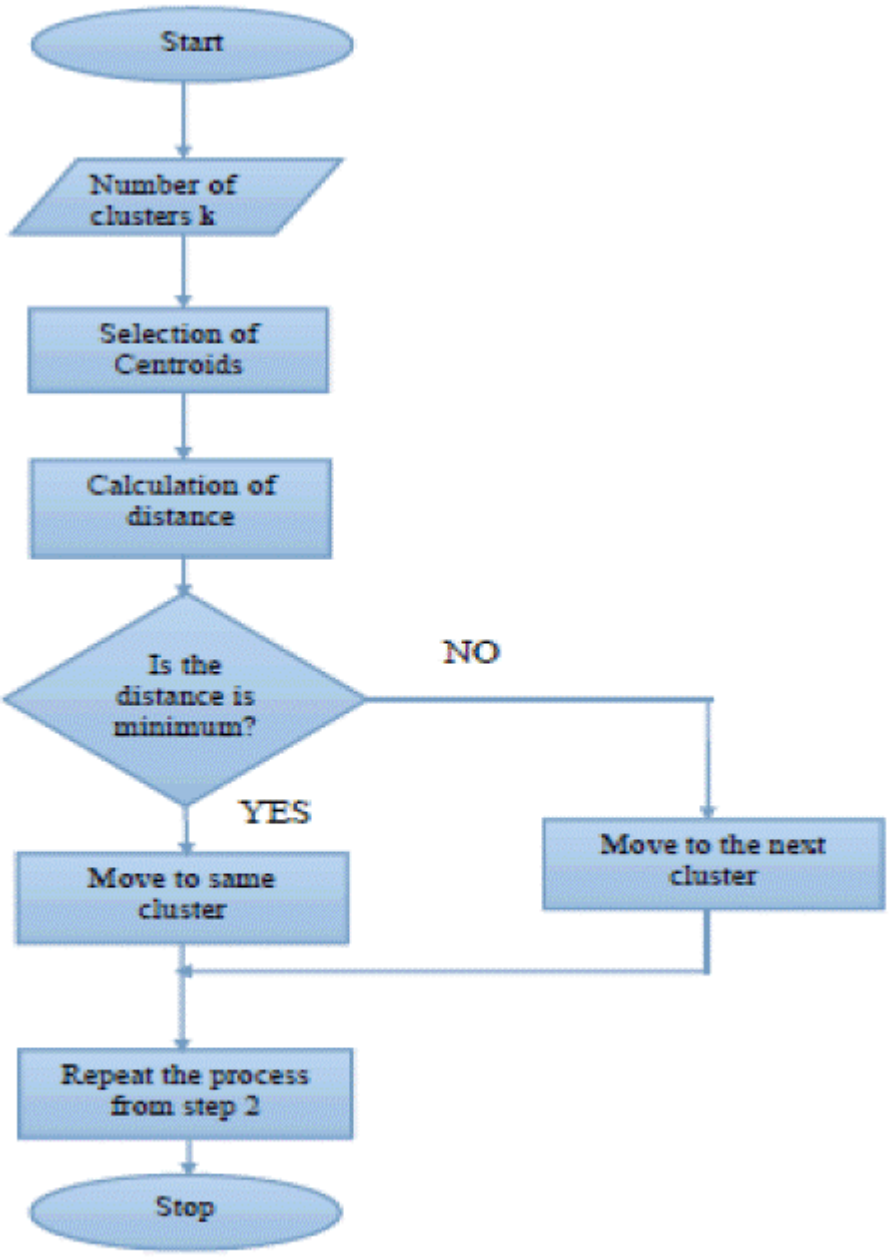


FIG. 1. **Flowchart for K-means clustering.**

Flowchart of K-Means Clustering

Understanding with a simple example

We will apply k-means on the following 1 dimensional data set for K=2.

Data set {2, 4, 10, 12, 3, 20, 30, 11, 25}

Iteration 1

M1, M2 are the two randomly selected centroids/means where

M1= 4, M2=11

and the initial clusters are

C1= {4}, C2= {11}

Calculate the Euclidean distance as

$D=[x,a]=\sqrt{(x-a)^2}$

D1 is the distance from M1

D2 is the distance from M2

Datapoint	D1	D2	Cluster
2	2	9	C1
4	0	7	C1
10	6	1	C2
12	8	1	C2
3	1	8	C1
20	16	9	C2
30	26	19	C2
11	7	0	C2
25	21	14	C2

Iteration 1

As we can see in the above table, 2 datapoints are added to cluster C1 and other datapoints added to cluster C2

Therefore

C1= {2, 4, 3}

C2= {10, 12, 20, 30, 11, 25}

Iteration 2

Calculate new mean of datapoints in C1 and C2.

Therefore

M1= (2+3+4)/3= 3

M2= (10+12+20+30+11+25)/6= 18

Calculating distance and updating clusters based on table below

Datapoint	D1	D2	Cluster
2	1	16	C1
4	1	14	C1
3	0	15	C1
10	7	8	C1
12	9	6	C2
20	17	2	C2
30	27	12	C2
11	8	7	C2
25	22	7	C2

Iteration 2

New Clusters

C1= {2, 3, 4, 10}

C2= {12, 20, 30, 11, 25}

Iteration 3

Calculate new mean of datapoints in C1 and C2.

Therefore

M1= (2+3+4+10)/4= 4.75

M2= (12+20+30+11+25)/5= 19.6

Calculating distance and updating clusters based on table below

Datapoint	D1	D2	Cluster
2	2.75	17.6	C1
4	0.75	15.6	C1
3	1.75	16.6	C1
10	5.25	9.6	C1
12	7.25	7.6	C1
20	15.25	0.4	C2
30	25.25	10.4	C2
11	6.25	8.6	C1
25	20.25	5.4	C2

Iteration 3

New Clusters

C1= {2, 3, 4, 10, 12, 11}

C2= {20, 30, 25}

Iteration 4

Calculate new mean of datapoints in C1 and C2.

Therefore

M1= (2+3+4+10+12+11)/6=7

M2= (20+30+25)/3= 25

Calculating distance and updating clusters based on table below

Datapoint	D1	D2	Cluster
2	5	23	C1
4	3	21	C1
3	4	22	C1
10	3	15	C1
12	5	13	C1
11	4	14	C1
20	13	5	C2
30	23	5	C2
25	18	0	C2

Iteration 4

New Clusters

C1= {2, 3, 4, 10, 12, 11}

C2= {20, 30, 25}

As we can see that the data points in the cluster C1 and C2 in iteration 3 are same as the data points of the cluster C1 and C2 of iteration 2.

It means that none of the data points has moved to other cluster. Also the means/centeroid of these clusters is constant. So this becomes the stopping condition for our algorithm.

How many clusters?

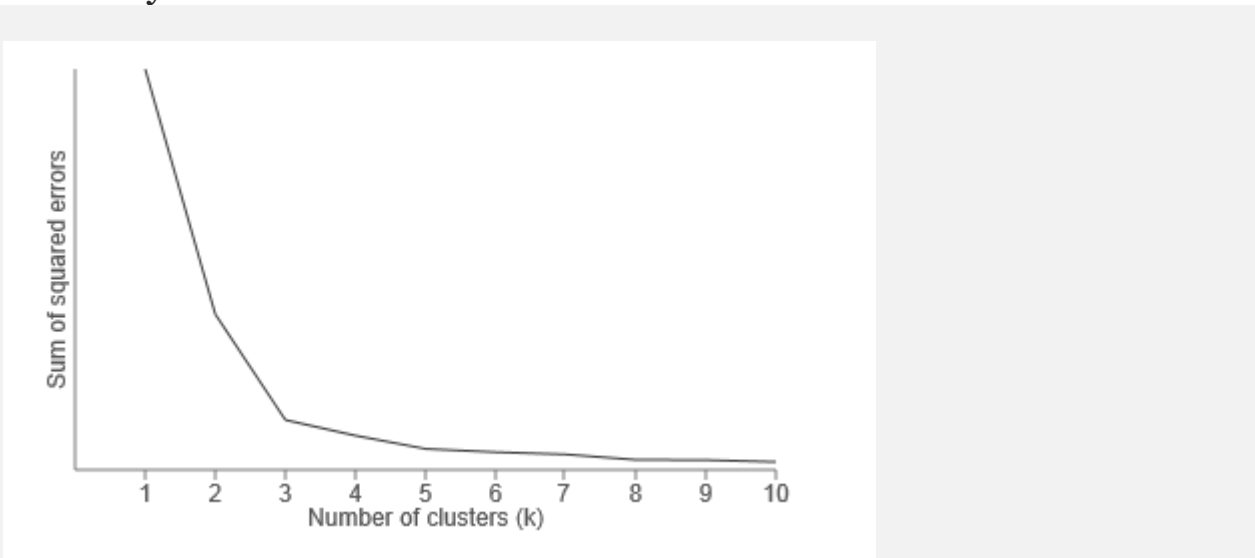
Selecting a proper value of ‘K’ is very difficult until we have a good knowledge about our data set.

Therefore we need some method to determine and validate weather we are using the right number of clusters. The fundamental aim of partitioning a data set is minimizing the intra-cluster variation or SSE. SSE can be calculated by –First take the difference between each data point with its centroid and then add up all the squares of the differences calculated

So to find optimal number of clusters:

Run k-means for different values of ‘K’. For example K varying from 1 to 10 and for each value of K compute SSE.

Plot a line chart K values on x axis and its corresponding values of SSE on y axis as shown below.



Elbow Method

SSE=0 if K=number of clusters, which means that each data point has its own cluster.

As we can see in the graph there is a rapid drop in SSE as we move from K=2 to 3 and it becomes almost constant as the value of K is further increased.

Because of the sudden drop we see an elbow in the graph. So the value to be considered for K is 3. This method is known as elbow method.

There are also many other techniques which are used to determine the value of K.

K-means is the ‘go-to’ clustering algorithm because it is fast and easy to understand.

Listing some drawbacks of K-Means

1. The result might not be globally optimal: We can’t assure that this algorithm will lead to the best global solution. Selecting different random seeds at the beginning affects the final results.

2. Value of K need to be specified beforehand: We can expect this value only if we have a good idea about our dataset and if we are working with a new dataset then elbow method can be used to determine value of K.

3. Works only for linear boundaries: K-means makes this assumption that the boundaries will be always linear. Hence it fails when it comes to complicated boundaries.

4. Slow for large number of samples: As this algorithm access each point of the dataset, it becomes slow when the sample size grows.

So this was all about K-Means. I hope now you have the better understanding of how k-means actually works. There are many other algorithms that are used for clustering in the industry.

K-Means Clustering Example:

Example:

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

Step 1:

Initialization: Randomly we choose following two centroids (k=2) for two clusters.

In this case the 2 centroid are: m1=(1.0,1.0) and m2=(5.0,7.0).

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

	Individual	Mean Vector
Group 1	1	(1.0, 1.0)
Group 2	4	(5.0, 7.0)

Step 2:

- Thus, we obtain two clusters containing:

  {1,2,3} and {4,5,6,7}.
- Their new centroids are:

$$m_1 = \left(\frac{1}{3}(1.0+1.5+3.0), \frac{1}{3}(1.0+2.0+4.0)\right) = (1.83, 2.33)$$

$$m_2 = \left(\frac{1}{4}(5.0+3.5+4.5+3.5), \frac{1}{4}(7.0+5.0+5.0+4.5)\right)$$

$$= (4.12, 5.38)$$

Individual	Centroid 1	Centroid 2
1	0	7.21
2 (1.5, 2.0)	1.12	6.10
3	3.61	3.61
4	7.21	0
5	4.72	2.5
6	5.31	2.06
7	4.30	2.92

$$d(m_1, 2) = \sqrt{|1.0-1.5|^2 + |1.0-2.0|^2} = 1.12$$

$$d(m_2, 2) = \sqrt{|5.0-1.5|^2 + |7.0-2.0|^2} = 6.10$$

Step 3:

- Now using these centroids we compute the Euclidean distance of each object, as shown in table.

Individual	Centroid 1	Centroid 2
1	1.57	5.38
2	0.47	4.28
3	2.04	1.78
4	5.64	1.84
5	3.15	0.73
6	3.78	0.54
7	2.74	1.08

- Therefore, the new clusters are:

  {1,2} and {**3**,4,5,6,7}

- Next centroids are: m1=(1.25,1.5) and m2 = (3.9,5.1)

- Step 4 :

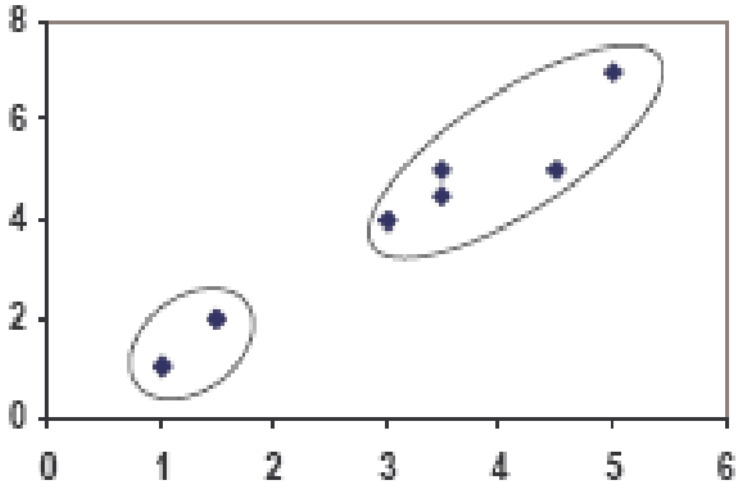
  The clusters obtained are:

  {1,2} and {3,4,5,6,7}

- Therefore, there is no change in the cluster.
- Thus, the algorithm comes to a halt here and final result consist of 2 clusters {1,2} and {3,4,5,6,7}.

Individual	Centroid 1	Centroid 2
1	0.56	5.02
2	0. 56	3.92
3	3.05	1.42
4	6.66	2.20
5	4.16	0.41
6	4.78	0.61
7	3.75	0.72

PLOT

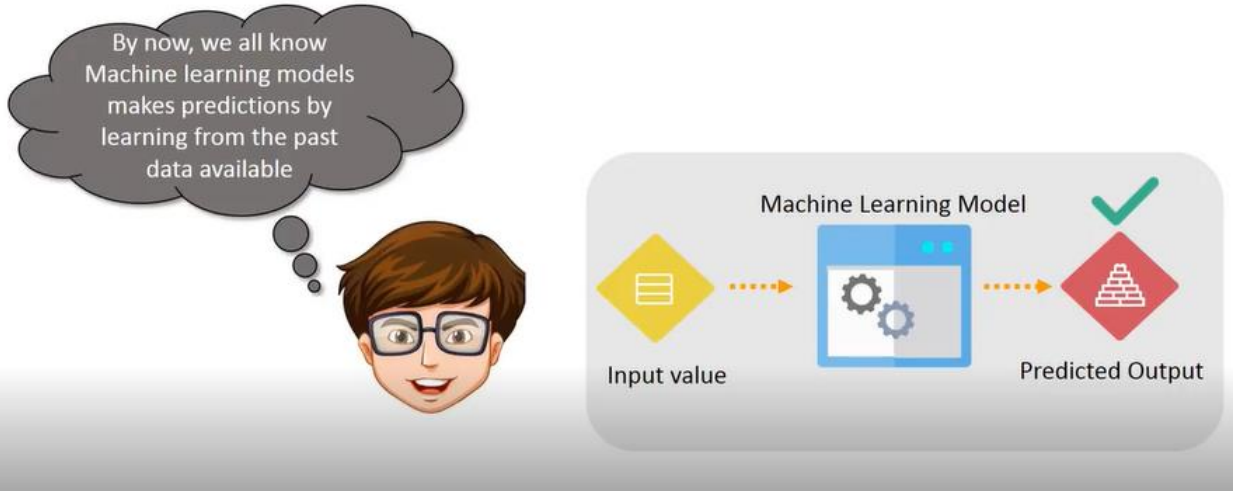


K-Nearest Neighbor Algorithm

- Why do we need KNN?
- What is KNN?
- How do we choose the factor 'K'?
- When do we use KNN?
- How does KNN Algorithm work?

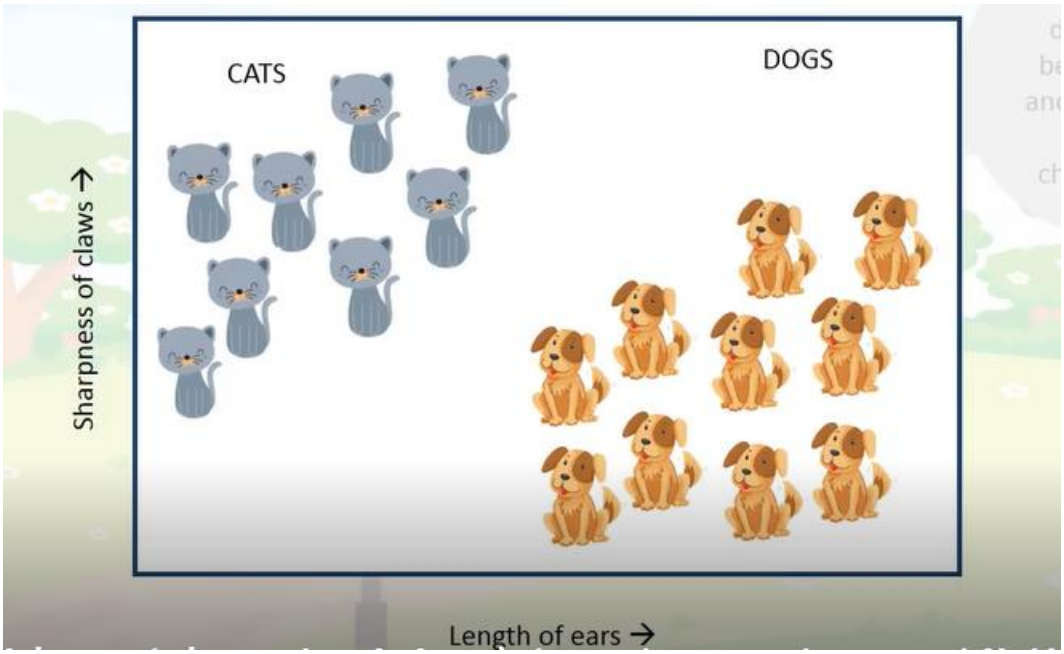


Why KNN?

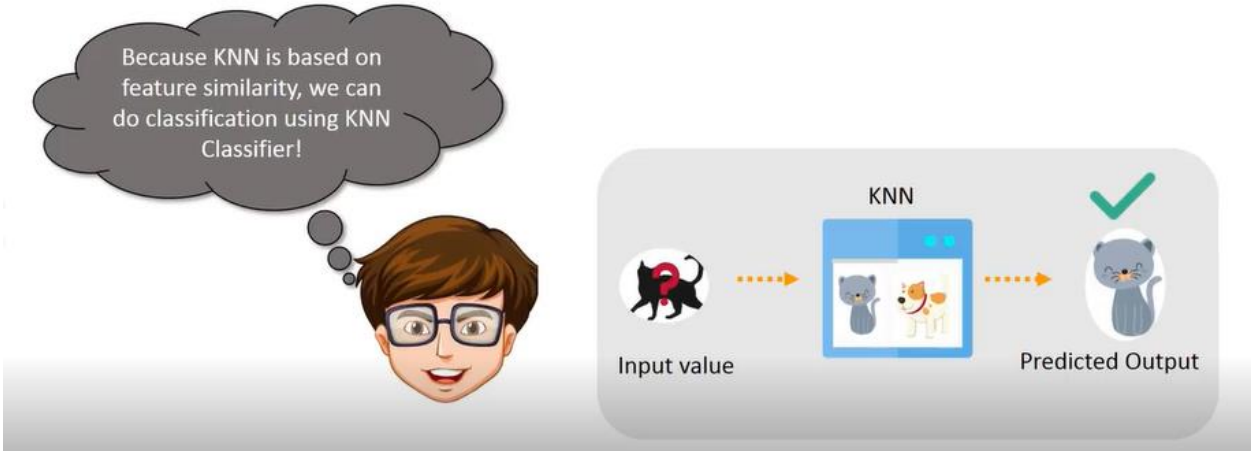




CATS	DOGS
	
Sharp Claws, uses to climb	Dull Claws
Smaller length of ears	Bigger length of ears
Meows and purrs	Barks
Doesn't love to play around	Loves to run around


CATS
DOGS
Sharpness of claws →
Length of ears →

Why KNN?


Because KNN is based on feature similarity, we can do classification using KNN Classifier!
KNN
Input value
Predicted Output

What is KNN Algorithm?

KNN – K Nearest Neighbors, is one of the simplest **Supervised** Machine Learning algorithm mostly used for

Classification

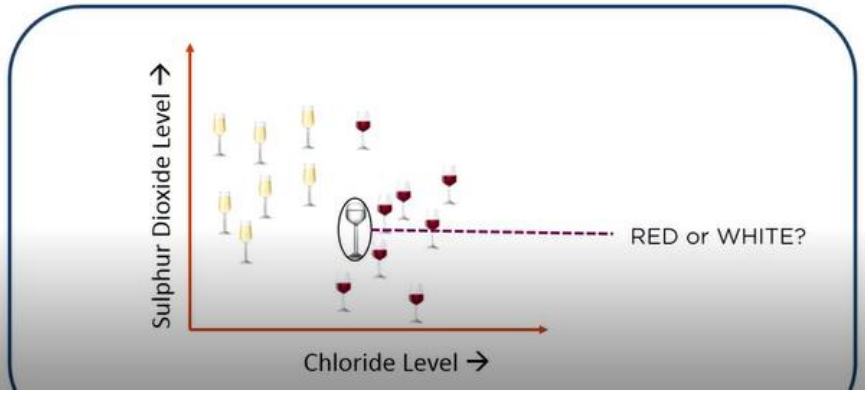


It classifies a data point based on how its neighbors are classified

What is KNN Algorithm?

KNN stores all available cases and classifies new cases based on a similarity measure

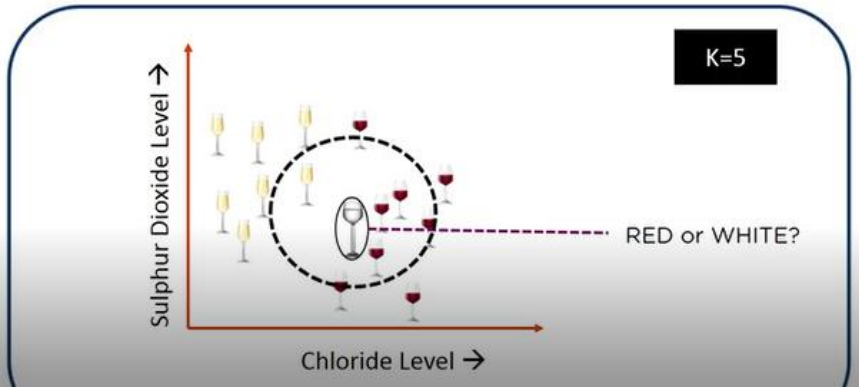




What is KNN Algorithm?

k in **KNN** is a parameter that refers to the number of nearest neighbors to include in the majority voting process

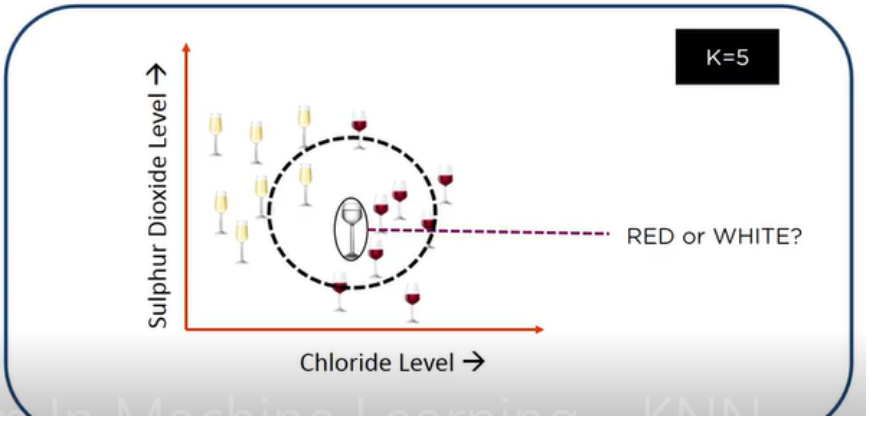




What is KNN Algorithm?

A data point is classified by majority votes from its 5 nearest neighbors

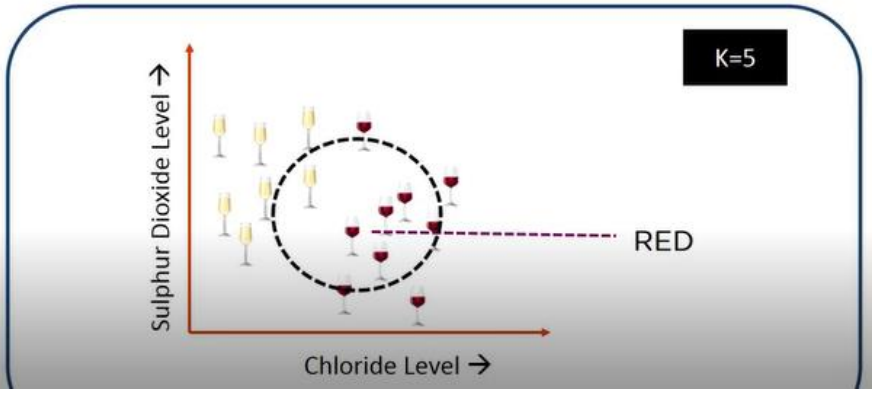




What is KNN Algorithm?

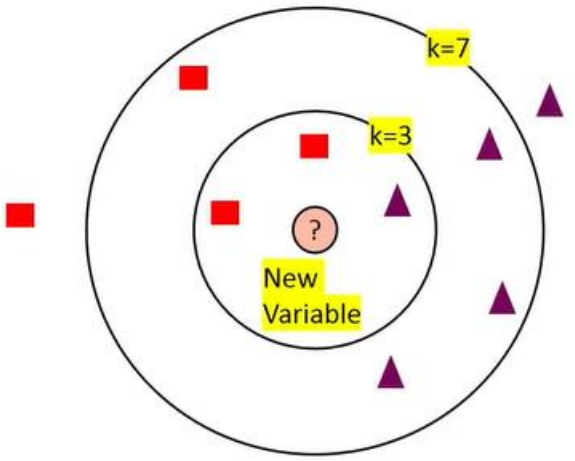
Here, the unknown point would be classified as red, since 4 out of 5 neighbors are red



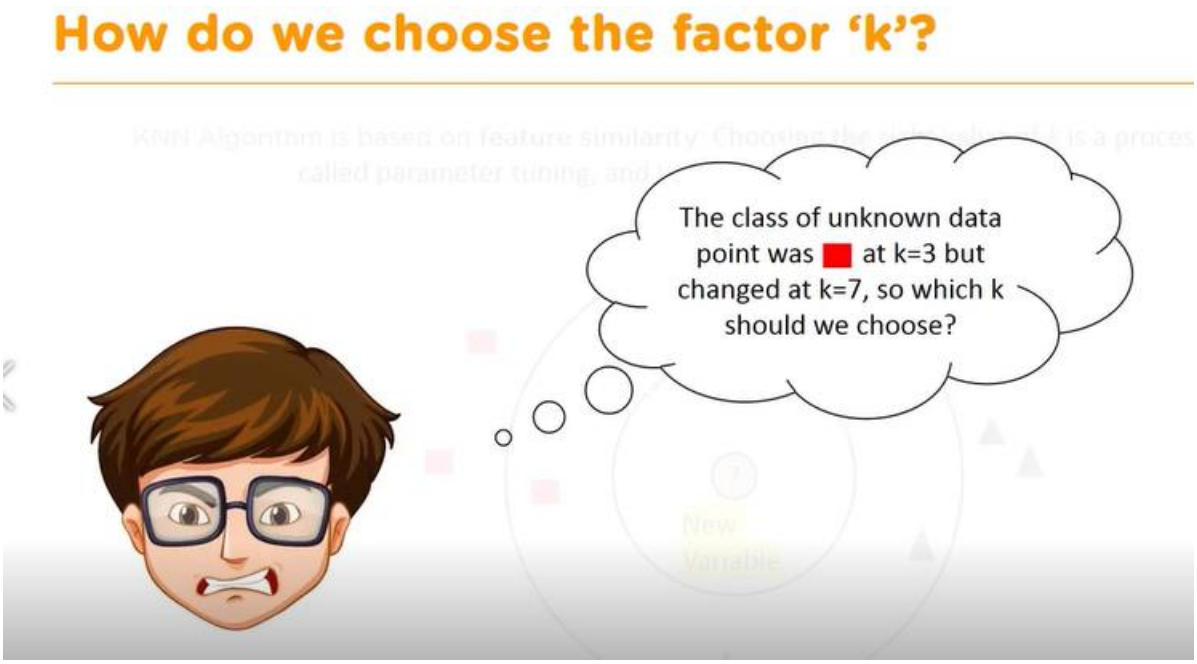


How do we choose the factor 'k'?

KNN Algorithm is based on **feature similarity**: Choosing the right value of *k* is a process called parameter tuning, and is important for better accuracy



But at k=7, we classify '?' as ▲



If K is too low, the bias is there, too noisy , might be skewed answer.

In KNN, finding the value of k is not easy. A small value of k means that noise will have a higher influence on the result and a large value make it computationally expensive. Data scientists usually choose as an odd number if the number of classes is 2 and another simple approach to select k is set k=sqrt(n).

Then how to select the optimal K value?

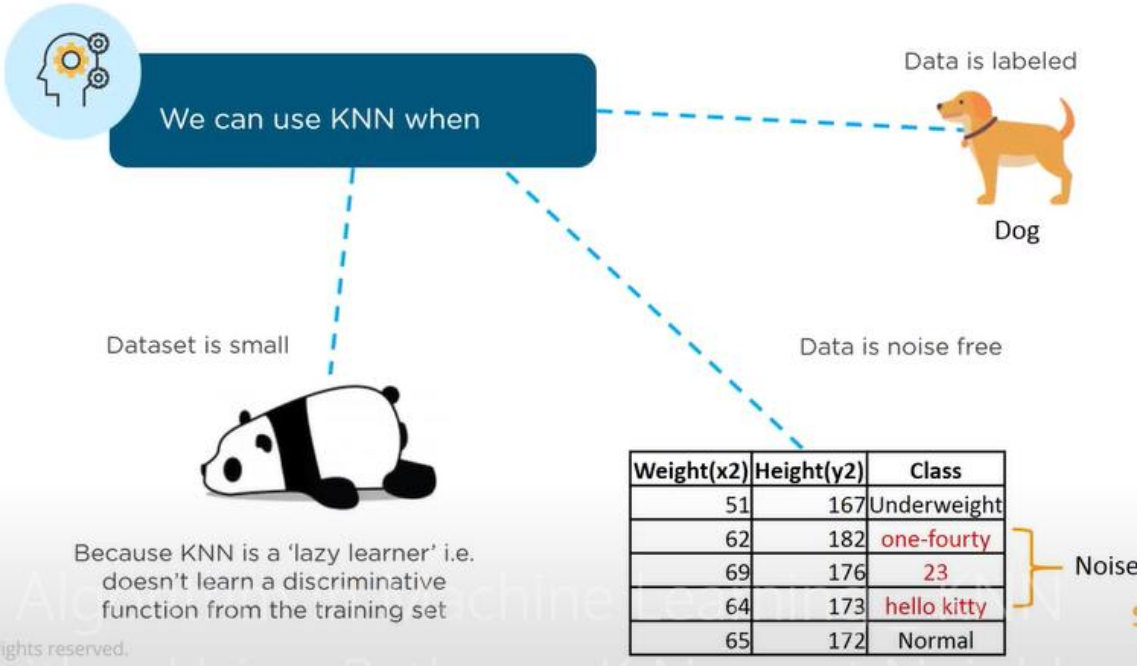
- There are no pre-defined statistical methods to find the most favorable value of K.
- Initialize a random K value and start computing.
- Choosing a small value of K leads to unstable decision boundaries.
- The substantial K value is better for classification as it leads to smoothening the decision boundaries.
- **Derive a plot between error rate and K denoting values in a defined range. Then choose the K value as having a minimum error rate.**

How do we choose the factor 'k'?

To choose a value of k:

- Sqrt(n), where n is the total number of data points
- Odd value of K is selected to avoid confusion between two classes of data

When do we use KNN Algorithm?



plilearn. All rights reserved.

Algorithm

Step-1: Select the number K of the neighbors

Step-2: Calculate the Euclidean distance of **K number of neighbors**

Step-3: Take the K nearest neighbors as per the calculated Euclidean distance.

Step-4: Among these k neighbors, count the number of the data points in each category.

Step-5: Assign the new data points to that category for which the number of the neighbor is maximum.

Step-6: Our model is ready.

How does KNN Algorithm work?



Consider a dataset having two variables: height (cm) & weight (kg) and each point is classified as Normal or Underweight

Weight(x2)	Height(y2)	Class
51	167	Underweight
62	182	Normal
69	176	Normal
64	173	Normal
65	172	Normal
56	174	Underweight
58	169	Normal
57	173	Normal
55	170	Normal

How does KNN Algorithm work?

On the basis of the given data we have to classify the below set as Normal or Underweight using KNN

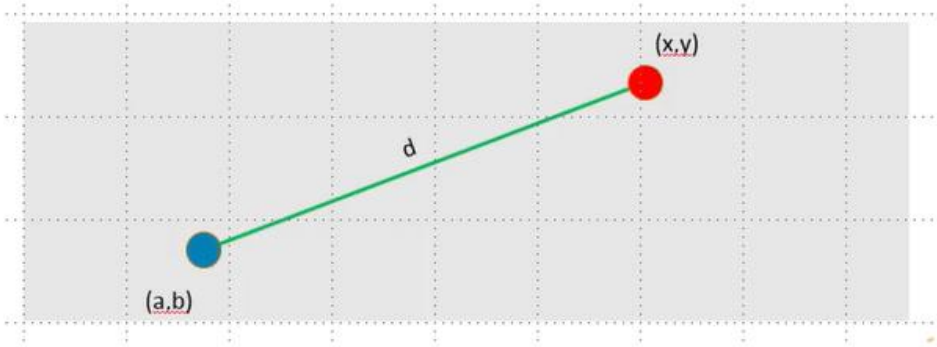
57 kg	170 cm	?



How does KNN Algorithm work?

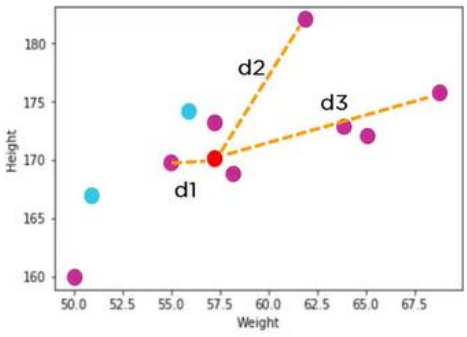
According to the **Euclidean distance** formula, the **distance** between two points in the plane with coordinates (x, y) and (a, b) is given by:

$$dist(d) = \sqrt{(x - a)^2 + (y - b)^2}$$



How does KNN Algorithm work?

Let's calculate it to understand clearly:



Unknown data point

dist(**d1**)= √(170-167)² + (57-51)² ~= 6.7

dist(**d2**)= √(170-182)² + (57-62)² ~= 13

dist(**d3**)= √(170-176)² + (57-69)² ~= 13.4

Similarly, we will calculate Euclidean distance of unknown data point from all the points in the dataset

How does KNN Algorithm work?

Hence, we have calculated the Euclidean distance of unknown data point from all the points as shown:

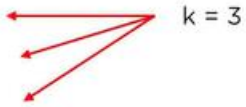
Where (x1, y1) = (57, 170) whose class we have to classify

Weight(x2)	Height(y2)	Class	Euclidean Distance
51	167	Underweight	6.7
62	182	Normal	13
69	176	Normal	13.4
64	173	Normal	7.6
65	172	Normal	8.2
56	174	Underweight	4.1
58	169	Normal	1.4
57	173	Normal	3
55	170	Normal	2

How does KNN Algorithm work?

Now, lets calculate the nearest neighbor at k=3

Weight(x2)	Height(y2)	Class	Euclidean Distance
51	167	Underweight	6.7
62	182	Normal	13
69	176	Normal	13.4
64	173	Normal	7.6
65	172	Normal	8.2
56	174	Underweight	4.1
58	169	Normal	1.4
57	173	Normal	3
55	170	Normal	2

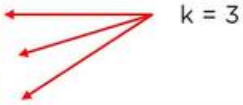


57 kg	170 cm	?

How does KNN Algorithm work?



Class	Euclidean Distance
Underweight	6.7
Normal	13
Normal	13.4
Normal	7.6
Normal	8.2
Underweight	4.1
Normal	1.4
Normal	3
Normal	2



So, majority neighbors are pointing towards '*Normal*'

Hence, as per KNN algorithm the class of (57, 170) should be 'Normal'

PRACTICE QUESTION

Given dataset:

Sepal Length	Sepal Width	Species
5.3	3.7	Setosa
5.1	3.8	Setosa
7.2	3.0	Virginica
5.4	3.4	Setosa
5.1	3.3	Setosa
5.4	3.9	Setosa
7.4	2.8	Virginica
6.1	2.8	Verscicolor
7.3	2.9	Virginica
6.0	2.7	Verscicolor
5.8	2.8	Virginica
6.3	2.3	Verscicolor
5.1	2.5	Verscicolor
6.3	2.5	Verscicolor
5.5	2.4	Verscicolor

Apply KNN classifier to classify the following sample.

Sepal Length	Sepal Width	Species
5.2	3.1	?

The Method of Least Squares

The Method of Least Squares: We come across variables during time series analysis, and many of them are the dependent type. Determining a relation between two or more variables is frequently required. The method of least squares is used to determine the best fit of a set of data points. The sum of the regressions of points from the plotted curve is minimised. It provides the best-fit trend line. In time series analysis, this method is the most widely used.

The least squares regression is utilised to predict the behaviour of dependent variables. The least squares method is widely used in evaluation and regression. This method is said to be a standard approach in regression analysis for approximating sets of equations with more equations than unknowns. It is associated with regression analysis and is widely used to make scatter plots easier to interpret.

What is the Method of Least Squares?

The method of least squares is a statistical method for determining the best fit line for given data in the form of an equation such as y=mx+b. The regression line is the curve of the equation. **The goal of this method is to minimise the sum of squared errors as much as possible.** This method is frequently used in data fitting, where the best fit result is supposed to reduce the sum of squared errors, which is defined as the difference between observed and fitted values. The sum of squared errors helps identify variation in observable data. The method of least squares analysis begins with a set of data points to be plotted on the graph of the XY plane.

MultiLinear Regression
y=m1x1+ m2x2+ m3x3+……..+b

Need for Least Square Method

Regression and evaluation make extensive use of the method of least squares. It is a conventional approach for the least square approximation of a set of equations with unknown variables than equations in the regression analysis procedure.

This method is used as a solution to minimise the sum of squares of all deviations each equation produces. It is commonly used in data fitting to reduce the sum of squared residuals of the discrepancies between the approximated and corresponding fitted values.

Line of Best Fit

The equation that gives the picture of the relationship between the data points is found in the line of best fit. It is calculated by the method of least squares. Computer software models that offer a summary of output values for analysis. The coefficients and summary output values explain the dependence of the variables being evaluated. It can be used to identify the line of best-fit equations.

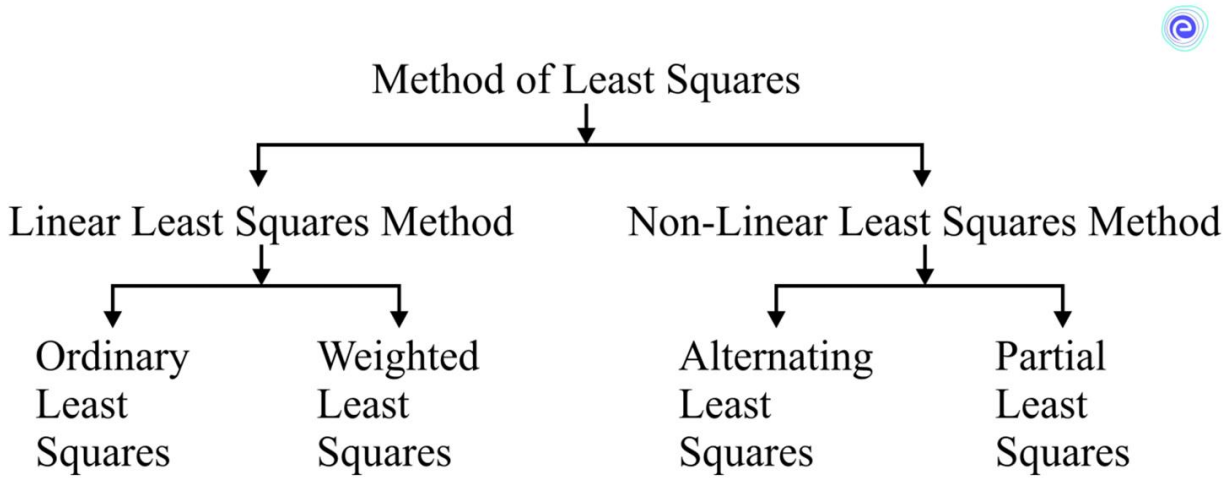
Least Squares Regression Line

A least squares regression line best fits a linear relationship between two variables by minimising the vertical distance between the data points and the regression line. Since it is the minimum value of the sum of squares of errors, it is also known as "variance," and the term "least squares" is also used.

On the vertical y-axis, the dependent variables are plotted, while the independent variables are plotted on the horizontal x-axis.

Types of Method of Least Squares

The method of least squares problems is divided into two categories. Linear or ordinary least square method and non-linear least square method. These are further classified as ordinary least squares, weighted least squares, alternating least squares and partial least squares.

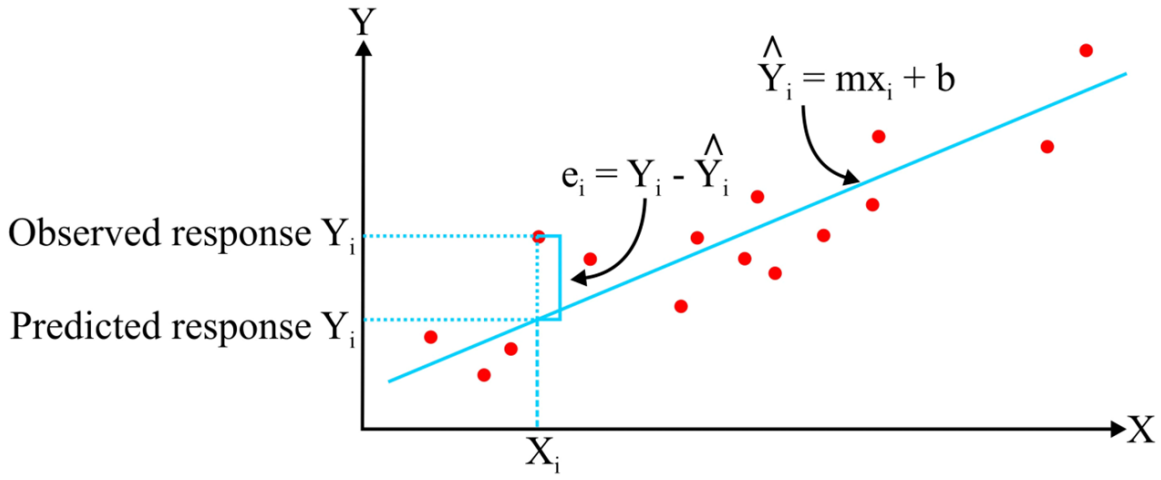


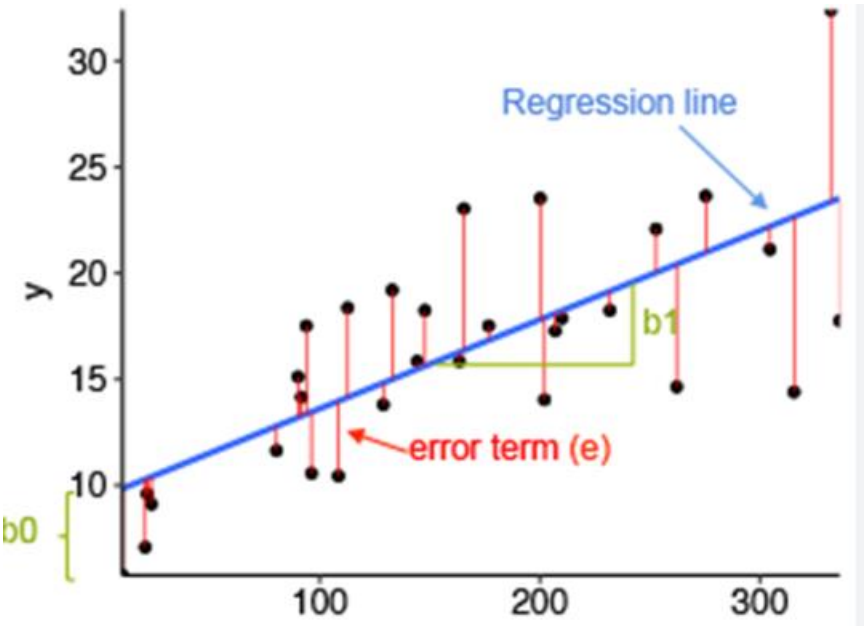
These are dependent on the residuals' linearity or non-linearity. In statistics, linear problems are frequently encountered in regression analysis. Non-linear problems are commonly used in the iterative refinement method.

Method of Least Squares Graph

Taking a look at the graph below, the straight line indicates a possible relationship between the independent and dependent variables. This method aims to reduce the gap between the observed response and the regression line's expected response. The model fits better if there are fewer residuals. This method reduces the residuals of each point from the line that must be used to minimise the data points. There are two types of residuals: vertical and perpendicular. Perpendicular is utilised in general, while vertical is employed largely in polynomials and hyperplane problems, as shown in the graphic below.

Slope=(y2-y1)/(x2-x1)


Y
$\hat{Y}_i = mx_i + b$
$e_i = Y_i - \hat{Y}_i$
Observed response Y_i
Predicted response Y_i
X
X_i



Line of Best Fit Equation

The line of best fits gives a set of observations with the least sum of squared residuals, or errors is known as the least-square technique. Assume the data points are $(x_1, x_2), (x_2, y_2), (x_3, y_3) \ldots\ldots, (x_n, y_n)$, with all $x's$ being independent variables and all $y's$ being dependent variables.

The linear line with the formula $y = mx + b$, where y and x are variables, m represents the slope, and b represents the y-intercept is found by using this method.

The following is the formula for calculating slope m and the value of b : $m = \frac{n\sum xy - \sum y \Sigma x}{n\sum x^2 - (\sum x)^2}$ $b = \frac{\sum y - m\sum x}{n}$. Here, n is the data points number.

Steps to Find the Line of Best Fit

The steps involved in the method of least squares using the given formulas are as follows.

- Create a table with four columns, the first two of which are for x and y coordinates.
- Find xy and (x^2) in the next two columns.
- Calculate $\sum x$, $\sum y$, $\sum xy$, and $\sum (x)^2$
- Using the formula, calculate the value of slope m.; $m = \frac{n\sum xy - \sum y \sum x}{n\sum x^2 - (\sum x)^2}$
- Using the formula, find the value of b.; $b = \frac{\Sigma y - m\sum x}{n}$
- In the equation $y = mx + b$, substitute the values of m and b.

Limitations of the Method of Least Squares

Even though the method of least squares is regarded as an excellent method for determining the best fit line, it has several drawbacks.

- Only the relationship between the two variables is displayed using this method. None of the other causes or effects is taken into account.
- This approach is unreliable when data is not evenly distributed and is particularly susceptible to outliers. The results of the least squares analysis may be skewed due to this.

Solved Examples – The Method of Least Squares

Below are a few solved examples that can help in getting a better idea.

Consider the points: $(1,1)$, $(-2,-1)$ *and* $(3,2)$. *In the same graph, plot these points and the least-squares regression line.*

Ans: The value of $n = 3$

x	y	xy	x^2
-2	-1	2	4
1	1	1	1
3	2	6	9
$\sum x = 2$	$\sum y = 2$	$\sum xy = 9$	$\sum x^2 = 14$

Here, $m = \frac{n\sum xy - \sum y \sum x}{n\sum x^2 - (\sum x)^2}$

$m = \frac{[(3\times 9) - (2\times 2)]}{3\times 14 - 2^2}$

$= \frac{27-4}{42-4}$

$\therefore m = \frac{23}{38}$

$b = \frac{\Sigma y - m\sum x}{n}$

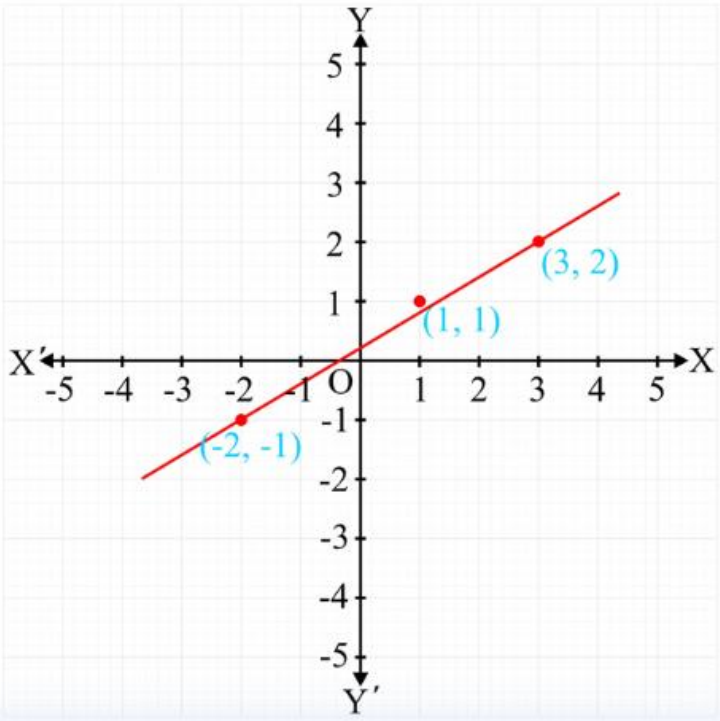
$b = \frac{\left[2-\left(\frac{23}{38}\right)\times 2\right]}{3}$

$= \frac{\left[2-\left(\frac{23}{19}\right)\right]}{3}$

$\therefore b = \frac{5}{19}$

So, the equation least squares regression line is $y = \frac{23}{38}x + \frac{5}{19}$

And, graph of the given points and the line of least squares is shown below:



Consider the following set of coordinates: $(-1,0)$, $(0,2)$, $(1,4)$, *and* $(k,5)$. *In the equation of least squares, the slope and* y*-intercept are* 1.7 *and* 1.9, **respectively.** *Can you find the value of* k**?**

Ans: From the given data, $n = 4$

Given:

$m = 1.7$

$b = 1.9$

x	y
-1	0
0	2
1	4
k	5
$\sum x = k$	$\sum y = 11$

View less

$b = \frac{\Sigma y - m\Sigma x}{n}$

$\Rightarrow 1.9 = \frac{11 - 1.7k}{4}$

$\Rightarrow 1.9 \times 4 = 11 - 1.7k$

$\Rightarrow 1.7k = 11 - 7.6$

$\Rightarrow k = \frac{3.4}{1.7}$

Question (iii):

. The following table shows the sales of a company (in million dollars)

x	2015	2016	2017	2018	2019
y	12	19	29	37	45

'mate the sales using the regression line in the year 2020.

Ans: From the given data, $n = 5$

Let us take $t = x - 2015$ (t is the number of years after 2015)

x	y	xy	x^2
0	12	0	0
1	19	19	1
2	29	58	4
3	37	111	9
4	45	180	16
$\sum x = 10$	$\sum y = 142$	$\sum xy = 368$	$\sum x^2 = 30$

$$m = \frac{n\Sigma xy - \Sigma y \Sigma x}{n\Sigma x^2 - (\sum x)^2}$$

$$m = \frac{[(5 \times 368) - (142 \times 10)]}{5 \times 30 - 10^2}$$

$$= \frac{1840 - 1420}{150 - 100}$$

$$= \frac{42}{5}$$

$$\therefore m = 8.4$$

$$b = \frac{\Sigma y - m\Sigma x}{n}$$

$$b = \frac{142 - 8.4 \times 10}{5}$$

$$= \frac{142 - 84}{5}$$

$$\therefore m = 11.6$$

Thus, the least squares equation is $y(t) = 8.4t + 11.6$

From the data, $t = 2020 - 2015$

$$\therefore t = 5$$

So, we can write,

$$y(5) = 8.4 \times 5 + 11.6$$

$$= 42 + 11.6$$

$$\therefore y(5) = 53.6$$

Hence, the number of sales is 53.6(million dollars) in 2020.

Question (iv):

Find the slope by using the method of least squares.

x	0	1	2	3	4
y	2	3	5	4	6

Question (v):

Find the intercept of the line of least squares.

x	2	4	6	8
y	3	5	7	9

Overfitting and Underfitting in Machine Learning

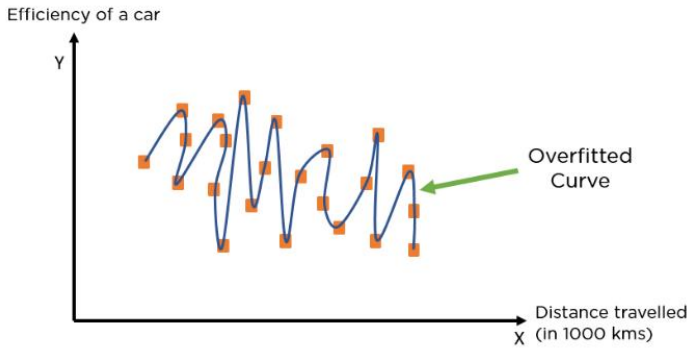
Before understanding the overfitting and underfitting, let's understand some basic term that will help to understand this topic well:

- **Signal:** It refers to the true underlying pattern of the data that helps the machine learning model to learn from the data.
- **Noise:** Noise is unnecessary and irrelevant data that reduces the performance of the model.
- **Bias:** Bias is a prediction error that is introduced in the model due to oversimplifying the machine learning algorithms. Or it is the difference between the predicted values and the actual values.
- **Variance:** If the machine learning model performs well with the training dataset, but does not perform well with the test dataset, then variance occurs.

What is Overfitting?

Overfitting occurs when our machine learning model tries to cover all the data points or more than the required data points present in the given dataset. Because of this, the model starts caching noise and inaccurate values present in the dataset, and all these factors reduce the efficiency and accuracy of the model.

When a model performs very well for training data but has poor performance with test data (new data), it is known as overfitting. In this case, the machine learning model learns the details and noise in the training data such that it negatively affects the performance of the model on test data. Overfitting can happen due to low bias and high variance.



Reasons for Overfitting

- Data used for training is not cleaned and contains noise (garbage values) in it

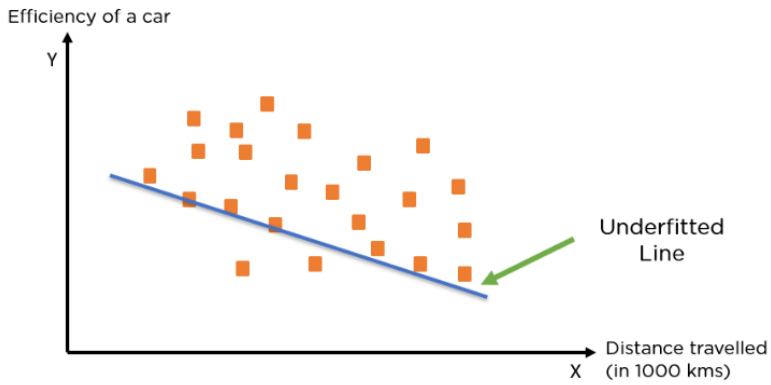
- The model has a high variance
- The size of the training dataset used is large
- The model is too complex

Ways to Tackle Overfitting

- Using K-fold cross-validation
- Using Regularization techniques such as Lasso and Ridge
- Training model with sufficient data
- Adopting ensembling techniques

What is Underfitting?

When a model has not learned the patterns in the training data well and is unable to generalize well on the new data, it is known as underfitting. An underfit model has poor performance on the training data and will result in unreliable predictions. Underfitting occurs due to high bias and low variance.



Reasons for Underfitting

- Data used for training is not cleaned and contains noise (garbage values) in it
- The model has a high bias
- The size of the training dataset used is not enough
- The model is too simple

Ways to Tackle Underfitting

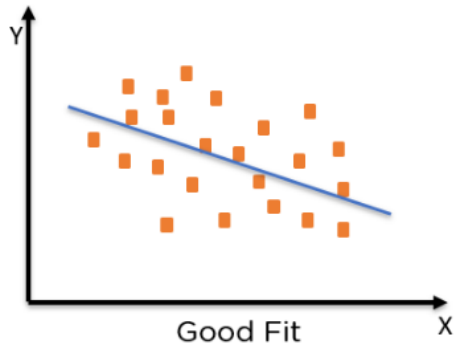
- Increase the number of features in the dataset
- Increase model complexity

- Reduce noise in the data
- Increase the duration of training the data

Now that you have understood what overfitting and underfitting are, let's see what is a good fit model in this tutorial on overfitting and underfitting in machine learning.

What Is a Good Fit In Machine Learning?

To find the good fit model, you need to look at the performance of a machine learning model over time with the training data. As the algorithm learns over time, the error for the model on the training data reduces, as well as the error on the test dataset. If you train the model for too long, the model may learn the unnecessary details and the noise in the training set and hence lead to overfitting. In order to achieve a good fit, you need to stop training at a point where the error starts to increase.



Hyperparameters in Machine Learning

Hyperparameters in Machine learning are those parameters that are explicitly defined by the user to control the learning process. These hyperparameters are used to improve the learning of the model, and their values are set before starting the learning process of the model.

In this topic, we are going to discuss one of the most important concepts of machine learning, i.e., Hyperparameters, their examples, hyperparameter tuning, categories of hyperparameters, how hyperparameter is different from parameter in Machine Learning? But before starting, let's first understand the Hyperparameter.

What are hyperparameters?

In Machine Learning/Deep Learning, a model is represented by its parameters. In contrast, a training process involves selecting the best/optimal hyperparameters that are used by learning algorithms to provide the best result. So, what are these hyperparameters? The answer is, "***Hyperparameters are defined as the parameters that are explicitly defined by the user to control the learning process.***"

Here the prefix "hyper" suggests that the parameters are top-level parameters that are used in controlling the learning process. The value of the Hyperparameter is selected and set by the machine learning engineer before the learning algorithm begins training the model. **Hence, these are external to the model, and their values cannot be changed during the training process**.

Some examples of Hyperparameters in Machine Learning

- The k in kNN or K-Nearest Neighbour algorithm
- Learning rate for training a neural network
- Train-test split ratio
- Batch Size
- Number of Epochs
- Branches in Decision Tree
- Number of clusters in Clustering Algorithm

Difference between Parameter and Hyperparameter?

There is always a big confusion between Parameters and hyperparameters or model hyperparameters. So, in order to clear this confusion, let's understand the difference between both of them and how they are related to each other.

Model Parameters:

Model parameters are configuration variables that are internal to the model, and a model learns them on its own. For example**, W Weights or Coefficients of independent variables in the Linear regression model**. or **Weights or Coefficients of independent variables in SVM, weight, and biases of a neural network, cluster centroid in clustering.** Some key points for model parameters are as follows:

- They are used by the model for making predictions.
- They are learned by the model from the data itself
- These are usually not set manually.
- These are the part of the model and key to a machine learning Algorithm.

Model Hyperparameters:

Hyperparameters are those parameters that are explicitly defined by the user to control the learning process. Some key points for model parameters are as follows:

- These are usually defined manually by the machine learning engineer.

- One cannot know the exact best value for hyperparameters for the given problem. The best value can be determined either by the rule of thumb or by trial and error.
- Some examples of Hyperparameters are **the learning rate for training a neural network, K in the KNN algorithm,**

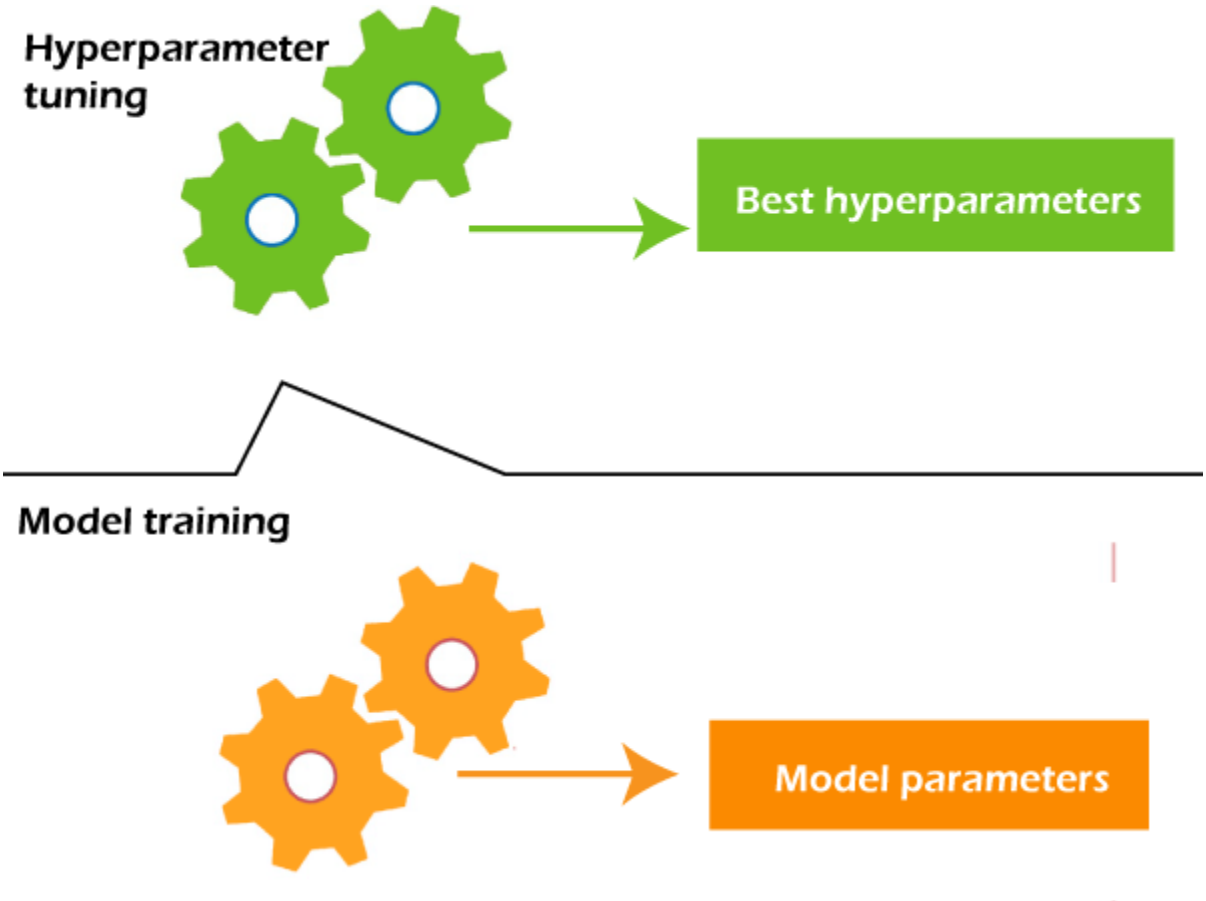
Categories of Hyperparameters

Broadly hyperparameters can be divided into two categories, which are given below:

1. **Hyperparameter for Optimization**
2. **Hyperparameter for Specific Models**

Hyperparameter for Optimization

The process of selecting the best hyperparameters to use is known as hyperparameter tuning, and the tuning process is also known as hyperparameter optimization. Optimization parameters are used for optimizing the model.



Some of the popular optimization parameters are given below:

- **Learning Rate:** The learning rate is the hyperparameter in optimization algorithms that controls how much the model needs to change in response to the estimated error for each time when the model's weights are updated. It is one of the crucial parameters while building a neural network, and also it determines the frequency of cross-checking with model parameters. Selecting the optimized learning rate is a challenging task because if the learning rate is very less, then it may slow down the training process. On the other hand, if the learning rate is too large, then it may not optimize the model properly.

Note: Learning rate is a crucial hyperparameter for optimizing the model, so if there is a requirement of tuning only a single hyperparameter, it is suggested to tune the learning rate.

- **Batch Size:** To enhance the speed of the learning process, the training set is divided into different subsets, which are known as a batch. **Number of Epochs:** An epoch can be defined as the complete cycle for training the machine learning model. Epoch represents an iterative learning process. The number of epochs varies from model to model, and various models are created with more than one epoch. To determine the right number of epochs, a validation error is taken into account. The number of epochs is increased until there is a reduction in a validation error. If there is no improvement in reduction error for the consecutive epochs, then it indicates to stop increasing the number of epochs.

Hyperparameter for Specific Models

Hyperparameters that are involved in the structure of the model are known as hyperparameters for specific models. These are given below:

- **A number of Hidden Units:** Hidden units are part of neural networks, which refer to the components comprising the layers of processors between input and output units in a neural network.

It is important to specify the number of hidden units hyperparameter for the neural network. It should be between the size of the input layer and the size of the output layer. More specifically, the number of hidden units should be 2/3 of the size of the input layer, plus the size of the output layer.

For complex functions, it is necessary to specify the number of hidden units, but it should not overfit the model.

- **Number of Layers:** A neural network is made up of vertically arranged components, which are called layers. There are mainly **input layers, hidden layers, and output layers**. A 3-layered neural network gives a better performance than a 2-layered network. For a Convolutional Neural network, a greater number of layers make a better model.

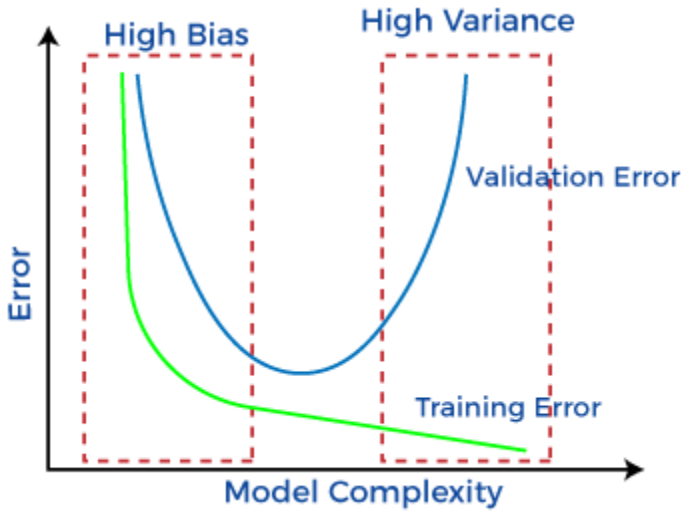
Bias and Variance

- Bias – error caused because the model can not represent the concept
- Variance – error caused because the learning algorithm overreacts to small changes (noise) in the training data

TotalLoss = Bias + Variance (+ noise)

Bias and Variance in Machine Learning

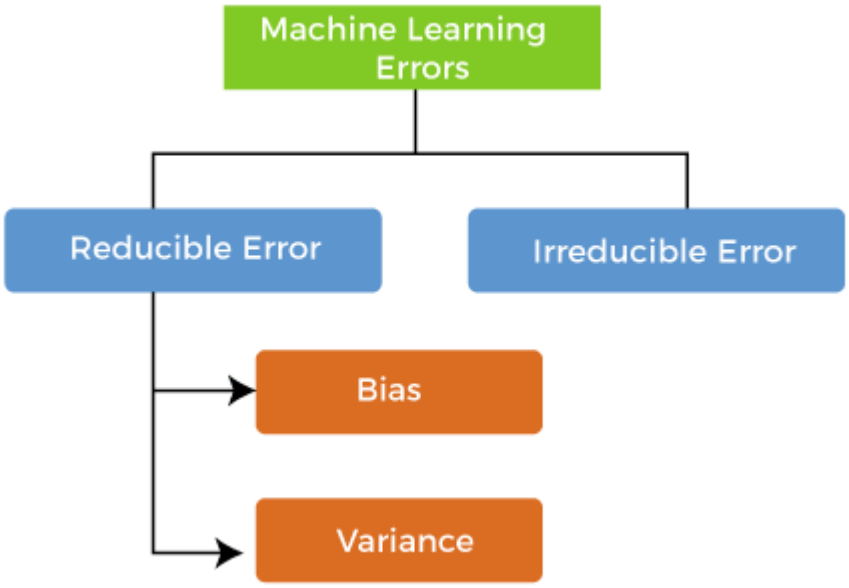
Machine learning is a branch of Artificial Intelligence, which allows machines to perform data analysis and make predictions. However, if the machine learning model is not accurate, it can make predictions errors, and these prediction errors are usually known as Bias and Variance. In machine learning, these errors will always be present as there is always a slight difference between the model predictions and actual predictions. The main aim of ML/data science analysts is to reduce these errors in order to get more accurate results. In this topic, we are going to discuss bias and variance, Bias-variance trade-off, Underfitting and Overfitting. But before starting, let's first understand what errors in Machine learning are?



Errors in Machine Learning?

In machine learning, an error is a measure of how accurately an algorithm can make predictions for the previously unknown dataset. On the basis of these errors, the machine learning model is selected that can perform best on the particular dataset. There are mainly two types of errors in machine learning, which are:

- **Reducible errors:** These errors can be reduced to improve the model accuracy. Such errors can further be classified into bias and Variance.



- **Irreducible errors:** These errors will always be present in the model regardless of which algorithm has been used. The cause of these errors is unknown variables whose value can't be reduced.

What is Bias?

In general, a machine learning model analyses the data, find patterns in it and make predictions. While training, the model learns these patterns in the dataset and applies them to test data for prediction. ***While making predictions, a difference occurs between prediction values made by the model and actual values/expected values, and this difference is known as bias errors or Errors due to bias***. It can be defined as an inability of machine learning algorithms such as Linear Regression to capture the true relationship between the data points. Each algorithm begins with some amount of bias because bias occurs from assumptions in the model, which makes the target function simple to learn. A model has either:

- **Low Bias:** A low bias model will make fewer assumptions about the form of the target function.
- **High Bias:** A model with a high bias makes more assumptions, and the model becomes unable to capture the important features of our dataset. **A high bias model also cannot perform well on new data.**

Generally, a linear algorithm has a high bias, as it makes them learn fast. The simpler the algorithm, the higher the bias it has likely to be introduced. Whereas a nonlinear algorithm often has low bias.

Some examples of machine learning algorithms with low bias **are Decision Trees, k-Nearest Neighbours and Support Vector Machines**. At the same time, an algorithm with high bias is **Linear Regression, Linear Discriminant Analysis and Logistic Regression.**

Ways to reduce High Bias:

High bias mainly occurs due to a much simple model. Below are some ways to reduce the high bias:

- Increase the input features as the model is underfitted.
- Decrease the regularization term.
- Use more complex models, such as including some polynomial features.

What is a Variance Error?

The variance would specify the amount of variation in the prediction if the different training data was used. In simple words, ***variance tells that how much a random variable is different from its expected value.*** Ideally, a model should not vary too much from one training dataset to another, which means the algorithm should be good in understanding the hidden mapping between inputs and output variables. Variance errors are either of **low variance or high variance.**

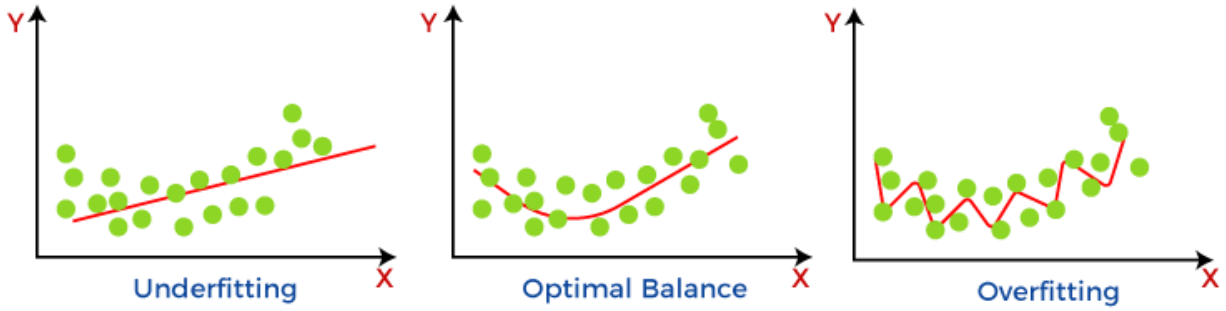
Low variance means there is a small variation in the prediction of the target function with changes in the training data set. At the same time, **High variance** shows a large variation in the prediction of the target function with changes in the training dataset.

A model that shows high variance learns a lot and perform well with the training dataset, and does not generalize well with the unseen dataset. As a result, such a model gives good results with the training dataset but shows high error rates on the test dataset.

Since, with high variance, the model learns too much from the dataset, it leads to overfitting of the model. A model with high variance has the below problems:

- A high variance model leads to overfitting.
- Increase model complexities.

Usually, nonlinear algorithms have a lot of flexibility to fit the model, have high variance.



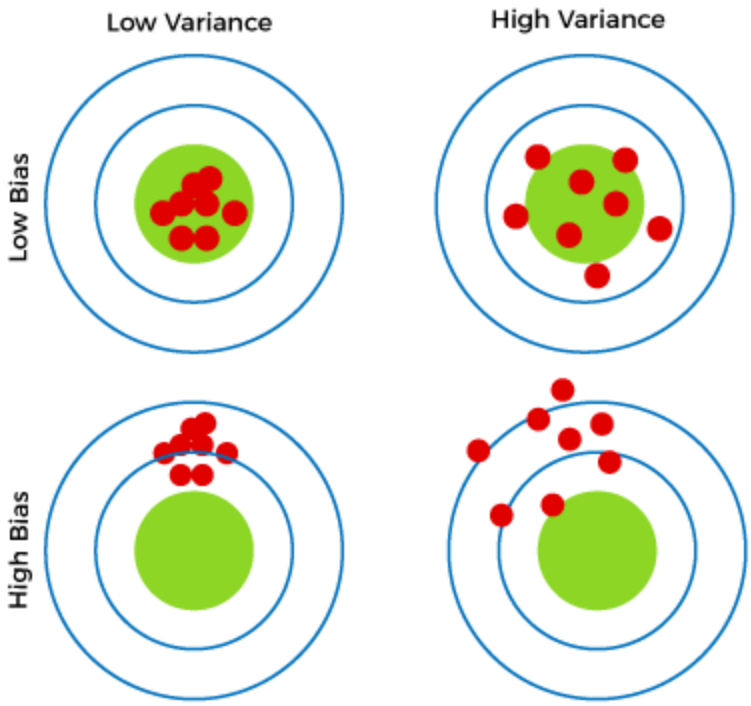
Some examples of machine learning algorithms with low variance are, **Linear Regression, Logistic Regression, and Linear discriminant analysis**. At the same time, algorithms with high variance are **decision tree, Support Vector Machine, and K-nearest neighbours.**

Ways to Reduce High Variance:

- Reduce the input features or number of parameters as a model is overfitted.
- Do not use a much complex model.
- Increase the training data.
- Increase the Regularization term.

Different Combinations of Bias-Variance

There are four possible combinations of bias and variances, which are represented by the below diagram:



1. **Low-Bias,Low-Variance:**
   The combination of low bias and low variance shows an ideal machine learning model. However, it is not possible practically.

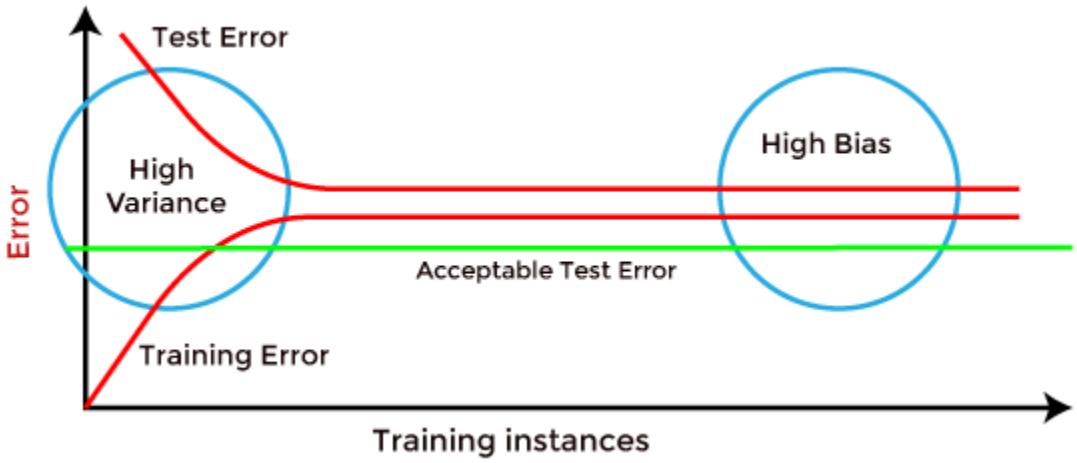
2. **Low-Bias, High-Variance:** With low bias and high variance, model predictions are inconsistent and accurate on average. This case occurs when the model learns with a large number of parameters and hence leads to an **overfitting**

3. **High-Bias, Low-Variance:** With High bias and low variance, predictions are consistent but inaccurate on average. This case occurs when a model does not learn well with the training dataset or uses few numbers of the parameter. It leads to **underfitting** problems in the model.

4. **High-Bias,High-Variance:**
   With high bias and high variance, predictions are inconsistent and also inaccurate on average.

How to identify High variance or High Bias?

High variance can be identified if the model has:



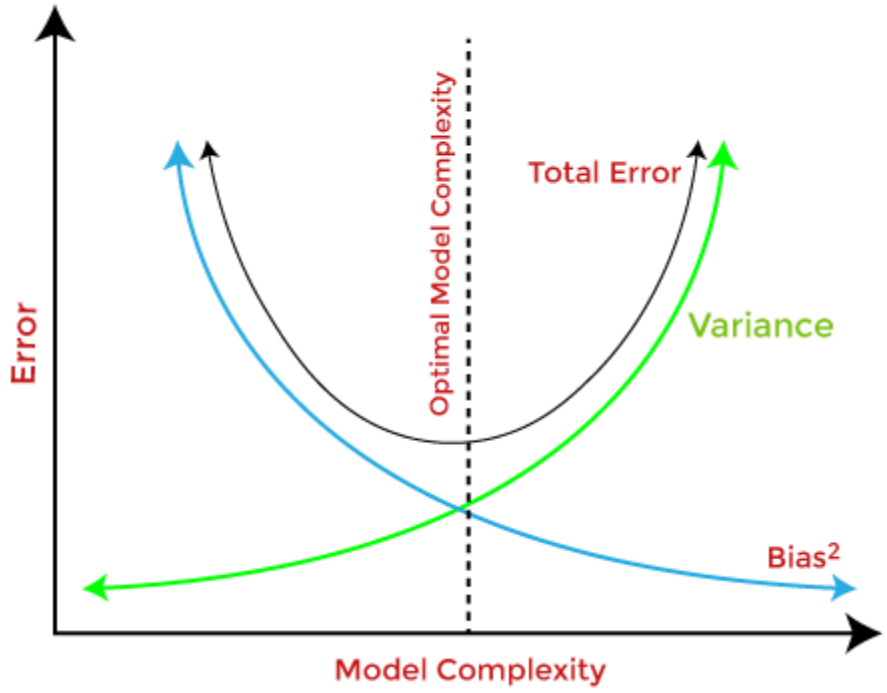
- Low training error and high test error.

High Bias can be identified if the model has:

- High training error and the test error is almost similar to training error.

Bias-Variance Trade-Off

While building the machine learning model, it is really important to take care of bias and variance in order to avoid overfitting and underfitting in the model. If the model is very simple with fewer parameters, it may have low variance and high bias. Whereas, if the model has a large number of parameters, it will have high variance and low bias. So, it is required to make a balance between bias and variance errors, and this balance between the bias error and variance error is known as **the Bias-Variance trade-off.**



For an accurate prediction of the model, algorithms need a low variance and low bias. But this is not possible because bias and variance are related to each other:

- If we decrease the variance, it will increase the bias.
- If we decrease the bias, it will increase the variance.

Bias-Variance trade-off is a central issue in supervised learning. Ideally, we need a model that accurately captures the regularities in training data and simultaneously generalizes well with the unseen dataset. Unfortunately, doing this is not possible simultaneously. Because a high variance algorithm may perform well with training data, but it may lead to overfitting to noisy data. Whereas, high bias algorithm generates a much simple model that may not even capture important regularities in the data. So, we need to find a sweet spot between bias and variance to make an optimal model.

Hence, the ***Bias-Variance trade-off is about finding the sweet spot to make a balance between bias and variance errors.***

Introduction to Dimensionality Reduction Technique

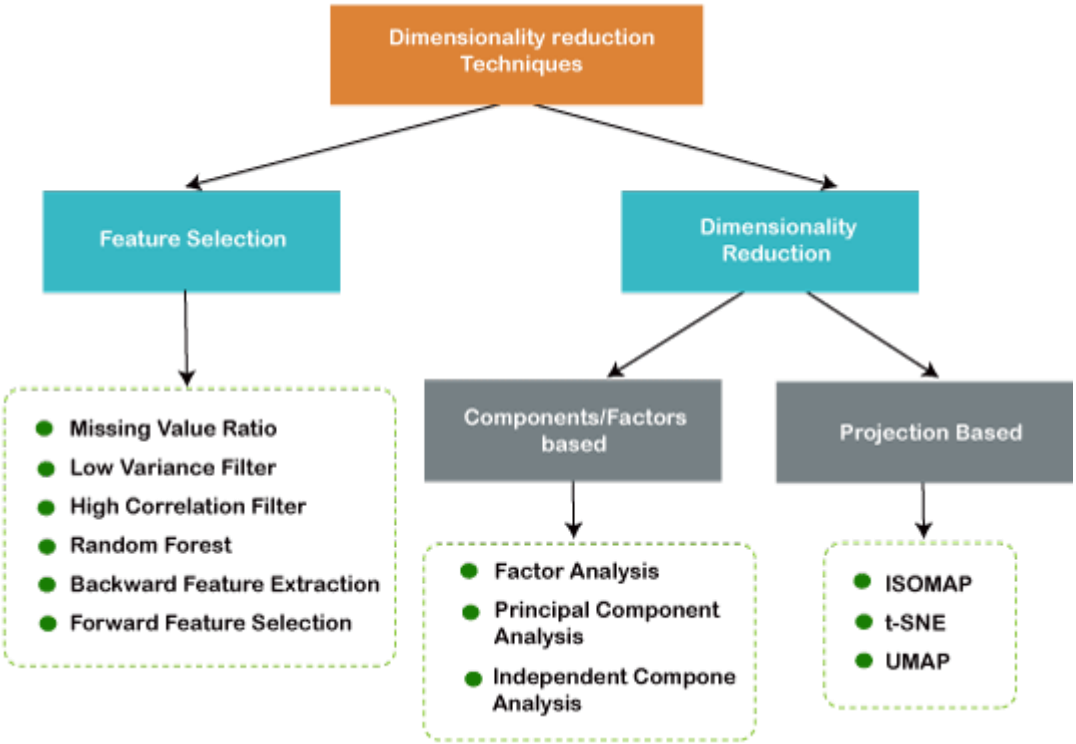
What is Dimensionality Reduction?

The number of input features, variables, or columns present in a given dataset is known as dimensionality, and the process to reduce these features is called dimensionality reduction.

A dataset contains a huge number of input features in various cases, which makes the predictive modeling task more complicated. Because it is very difficult to visualize or make predictions for the training dataset with a high number of features, for such cases, dimensionality reduction techniques are required to use.

Dimensionality reduction technique can be defined as, ***"It is a way of converting the higher dimensions dataset into lesser dimensions dataset ensuring that it provides similar information."*** These techniques are widely used in machine learning for obtaining a better fit predictive model while solving the classification and regression problems.

It is commonly used in the fields that deal with high-dimensional data, such as **speech recognition, signal processing, bioinformatics, etc. It can also be used for data visualization, noise reduction, cluster analysis**, etc.



Feature Engineering

Feature engineering is a crucial step in machine learning that involves preparing and transforming raw data into features that better represent the underlying problem to the predictive models. The main techniques are feature selection, feature extraction, and feature transformation.

Feature Selection--Choosing the most relevant features from the original set, eliminating redundant or irrelevant data.

Feature Extraction--Creating new features from the existing ones, often reducing the dimensionality of the dataset.

Feature Transformation--Changing the scale or distribution of features to make them more suitable for model learning.

Feature Transformation Techniques:
Normalization / Standardization
Example: Scaling all numerical features to have mean 0 and standard deviation 1 before training a logistic regression model.
Log, square root, or box-cox transformations
Example: Applying log transformation to skewed income data to make it more normally distributed.
Binning
Example: Converting ages into age groups (e.g., 18–25, 26–35) for decision trees.
One-hot encoding / Label encoding
Example: Turning a "color" categorical variable with values {red, green, blue} into one-hot encoded vectors.

The Curse of Dimensionality

Handling the high-dimensional data is very difficult in practice, commonly known as the *curse of dimensionality.* If the dimensionality of the input dataset increases, any machine learning algorithm and model becomes more complex. As the number of features increases, the number of samples also gets increased proportionally, and the chance of overfitting also increases. If the machine learning model is trained on high-dimensional data, it becomes overfitted and results in poor performance.

Hence, it is often required to reduce the number of features, which can be done with dimensionality reduction.

Benefits of applying Dimensionality Reduction

Some benefits of applying dimensionality reduction technique to the given dataset are given below:

- By reducing the dimensions of the features, the space required to store the dataset also gets reduced.
- Less Computation training time is required for reduced dimensions of features.
- Reduced dimensions of features of the dataset help in visualizing the data quickly.
- **I**t removes the redundant features (if present) by taking care of multicollinearity.

Disadvantages of dimensionality Reduction

There are also some disadvantages of applying the dimensionality reduction, which are given below:

- Some data may be lost due to dimensionality reduction.
- In the PCA dimensionality reduction technique, sometimes the principal components required to consider are unknown.

Approaches of Dimension Reduction

There are two ways to apply the dimension reduction technique, which are given below:

Feature Selection

Feature selection is the process of selecting the subset of the relevant features and leaving out the irrelevant features present in a dataset to build a model of high accuracy. In other words, it is a way of selecting the optimal features from the input dataset.

Three methods are used for the feature selection:

1. Filters Methods

In this method, the dataset is filtered, and a subset that contains only the relevant features is taken. Some common techniques of filters method are:

- **Correlation**
- **Chi-Square Test**
- **ANOVA**
- **Information Gain, etc.**

2. Wrappers Methods

The wrapper method has the same goal as the filter method, but it takes a machine learning model for its evaluation. In this method, some features are fed to the ML model, and evaluate the performance. The performance decides whether to add those features or remove to increase the accuracy of the model. This method is more accurate than the filtering method but complex to work. Some common techniques of wrapper methods are:

- Forward Selection
- Backward Selection
- Bi-directional Elimination

3. Embedded Methods: Embedded methods check the different training iterations of the machine learning model and evaluate the importance of each feature. Some common techniques of Embedded methods are:

- **LASSO**
- **Elastic Net**
- **Ridge Regression, etc.**

Feature Extraction:

Feature extraction is the process of transforming the space containing many dimensions into space with fewer dimensions. This approach is useful when we want to keep the whole information but use fewer resources while processing the information.

Some common feature extraction techniques are:

1. Principal Component Analysis
2. Linear Discriminant Analysis
3. Kernel PCA
4. Quadratic Discriminant Analysis

Common techniques of Dimensionality Reduction

1. **Principal Component Analysis**
2. **Backward Elimination**
3. **Forward Selection**
4. **Score comparison**
5. **Missing Value Ratio**
6. **Low Variance Filter**
7. **High Correlation Filter**
8. **Random Forest**
9. **Factor Analysis**
10. **Auto-Encoder**

Principal Component Analysis (PCA)

Principal Component Analysis is a statistical process that converts the observations of correlated features into a set of linearly uncorrelated features with the help of orthogonal transformation. These new

transformed features are called the **Principal Components**. It is one of the popular tools that is used for exploratory data analysis and predictive modeling.

PCA works by considering the variance of each attribute because the high attribute shows the good split between the classes, and hence it reduces the dimensionality. Some real-world applications of PCA are ***image processing, movie recommendation system, optimizing the power allocation in various communication channels.***

Backward Feature Elimination

The backward feature elimination technique is mainly used while developing Linear Regression or Logistic Regression model. Below steps are performed in this technique to reduce the dimensionality or in feature selection:

- In this technique, firstly, all the n variables of the given dataset are taken to train the model.
- The performance of the model is checked.
- Now we will remove one feature each time and train the model on n-1 features for n times, and will compute the performance of the model.
- We will check the variable that has made the smallest or no change in the performance of the model, and then we will drop that variable or features; after that, we will be left with n-1 features.
- Repeat the complete process until no feature can be dropped.

In this technique, by selecting the optimum performance of the model and maximum tolerable error rate, we can define the optimal number of features require for the machine learning algorithms.

Forward Feature Selection

Forward feature selection follows the inverse process of the backward elimination process. It means, in this technique, we don't eliminate the feature; instead, we will find the best features that can produce the highest increase in the performance of the model. Below steps are performed in this technique:

- We start with a single feature only, and progressively we will add each feature at a time.
- Here we will train the model on each feature separately.
- The feature with the best performance is selected.
- The process will be repeated until we get a significant increase in the performance of the model.

Missing Value Ratio

If a dataset has too many missing values, then we drop those variables as they do not carry much useful information. To perform this, we can set a threshold level, and if a variable has missing values more than

that threshold, we will drop that variable. The higher the threshold value, the more efficient the reduction.

Low Variance Filter

As same as missing value ratio technique, data columns with some changes in the data have less information. Therefore, we need to calculate the variance of each variable, and all data columns with variance lower than a given threshold are dropped because low variance features will not affect the target variable.

High Correlation Filter

High Correlation refers to the case when two variables carry approximately similar information. Due to this factor, the performance of the model can be degraded. This correlation between the independent numerical variable gives the calculated value of the correlation coefficient. If this value is higher than the threshold value, we can remove one of the variables from the dataset. We can consider those variables or features that show a high correlation with the target variable.

Random Forest

Random Forest is a popular and very useful feature selection algorithm in machine learning. This algorithm contains an in-built feature importance package, so we do not need to program it separately. In this technique, we need to generate a large set of trees against the target variable, and with the help of usage statistics of each attribute, we need to find the subset of features.

Random forest algorithm takes only numerical variables, so we need to convert the input data into numeric data using **hot encoding**.

Factor Analysis

Factor analysis is a technique in which each variable is kept within a group according to the correlation with other variables, it means variables within a group can have a high correlation between themselves, but they have a low correlation with variables of other groups.

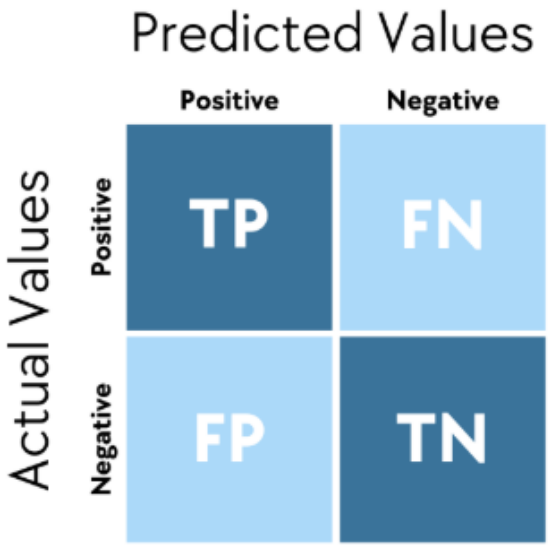
We can understand it by an example, such as if we have two variables Income and spend. These two variables have a high correlation, which means people with high income spends more, and vice versa. So, such variables are put into a group, and that group is known as the **factor**. The number of these factors will be reduced as compared to the original dimension of the dataset.

Auto-encoders

One of the popular methods of dimensionality reduction is auto-encoder, which is a type of ANN or artificial neural network, and its main aim is to copy the inputs to their outputs. In this, the input is compressed into latent-space representation, and output is occurred using this representation. It has mainly two parts:

- **Encoder:** The function of the encoder is to compress the input to form the latent-space representation.
- **Decoder:** The function of the decoder is to recreate the output from the latent-space representation.

A **confusion matrix** is a table used to evaluate the performance of a classification model (especially in binary classification). It shows the number of correct and incorrect predictions made by the model compared to the actual outcomes (truth).



Example--

In a spam detection system, out of 100 actual spam emails, 90 are correctly classified as spam. Out of 200 actual non-spam emails, 180 are correctly classified as non-spam.

Determine the confusion matrix. Then compute the accuracy, sensitivity (recall), specificity, precision, and F1 score. Comment on the performance of the spam detection system.

Confusion Matrix

	Predicted Spam	**Predicted Not Spam**
Actual Spam	90 (TP)	10 (FN)
Actual Not Spam	20 (FP)	180 (TN)

- **TP (True Positives)** = 90
- **FN (False Negatives)** = 10
- **FP (False Positives)** = 20
- **TN (True Negatives)** = 180

Metrics Calculation

Accuracy =

(TP+TN)/(TP+TN+FP+FN)

= (90 + 180) / (90 + 10 + 20 + 180) = 270 / 300 = **0.90**

Sensitivity (Recall / True Positive Rate) =

TP / (TP + FN)

= 90 / (90 + 10) = 90 / 100 = **0.90**

Specificity (True Negative Rate) =

TN / (TN + FP)

= 180 / (180 + 20) = 180 / 200 = **0.90**

Precision (Positive Predictive Value) =

TP / (TP + FP)

= 90 / (90 + 20) = 90 / 110 ≈ **0.818**

F1 Score =

2 × (Precision × Recall) / (Precision + Recall)

= 2 × (0.818 × 0.9) / (0.818 + 0.9) ≈ 2 × 0.736 / 1.718 ≈ **0.857**

Example-2

A fraud detection system correctly identifies 45 out of 50 fraudulent transactions and correctly classifies 940 out of 950 legitimate ones.

Determine the confusion matrix and compute accuracy, sensitivity, specificity, precision, and F1 score. Comment on performance.

ANS

Confusion Matrix:

	Predicted Fraud	**Predicted Legitimate**
Actual Fraud	45 (TP)	5 (FN)
Actual Legitimate	10 (FP)	940 (TN)

Accuracy = (45 + 940) / 1000 = **0.985 or 98.5%**

Sensitivity = 45 / (45 + 5) = **0.90**

Specificity = 940 / (940 + 10) = **0.989**

Precision = 45 / (45 + 10) = **0.818**

F1 Score = 2 × (0.818 × 0.90) / (0.818 + 0.90) ≈ **0.857**

Example-3

Out of 100 infected individuals, 92 test positive using a new COVID-19 kit. Out of 150 non-infected people, 140 test negative.

Determine the confusion matrix, compute accuracy, sensitivity, specificity, precision, and F1 score.

ANS

Confusion Matrix:

	Test Positive	**Test Negative**
Infected	92 (TP)	8 (FN)
Not Infected	10 (FP)	140 (TN)

Accuracy = (92 + 140) / 250 = **0.928 or 92.8%**

Sensitivity = 92 / (92 + 8) = **0.92**

Specificity = 140 / (140 + 10) = **0.933**

Precision = 92 / (92 + 10) = **0.902**

F1 Score = 2 × (0.902 × 0.92) / (0.902 + 0.92) ≈ **0.911**